Theory of Computer Science (TCS)

Module No.	Topics
1	Basic concepts & Finite Automata
2	Regular Expressions & Languages
3	Grammers
4	Pushdown Automata (PDA)
5	Turing Machine (TM)
6	Undecidability

Module 1 - Basic Concepts & Finite Automata

* Alphabets, Strings, Languages -

① **Alphabet** - An alphabet is a finite collection of symbols

Ex - i) $S = \{a, b, c, \ldots z\}$

where a, b, c .... z are alphabets denoted by S

ii) $W = \{0, 1\}$

where 0, 1 are aphabets denoted by W

② **String** - A string is a finite collection of symbols from alphabet

-. If $\Sigma = \{a, b\}$ then various strings that can be formed from Σ are $\{ab, aa, aaa, bb, bbb, ba \ldots\}$

-. Finite ...

① Length of string - It is denoted by $|\omega|$ & it is a count of no. of symbols of which ω is made up of

Ex - If $\omega = XYZ$ then $|\omega| = 3$

If $|\omega| = 0$ then the string is called as empty string & we use notation ε to denote empty string

② Prefix of string - A Prefix of string is the string formed by taking any numbers of leading symbols of string.

Ex - If $\omega = abc$ then ε, a, ab, abc are the Prefixes of ω

If $|\omega| = n$, then there exist $n+1$ Prefixes of ω

Suffix of String - A suffix of a string is a string formed by taking any no. of trailing symbols of a string.

Ex - If $\omega = abc$ then ε, c, bc, abc are the suffixes of a string ω

If $|\omega| = n$ then there exist $n+1$ suffixes of ω

④ **Substring** - A substring of a string is the string formed by taking any no. of symbols of string

Ex - If $\omega = abc$ then $\varepsilon, a, b, c, ab, bc$ & abc are the substrings of the ω.

⑤ **Subsequence** - A subsequence of a string is the string formed by taking any no. of symbols of string, that need not necessarily be consecutive.

Ex - If $\omega = abc$ then $\varepsilon, a, b, c, ab, bc, ac$ & abc are the subsequences of the ω

③ Language – It is a set of string formed by using the symbols belonging to some particular chosen alphabet

Ex - (i) If $\Sigma = \{0, 1\}$ then one of the language that can be defined over this Σ will be

$L = \{\varepsilon, 0, 00, 000, 1, 11, 111 \ldots\}$

(ii) If $\Sigma = \{\varepsilon, 0, 00, 000 \ldots\}$

Here Language is defined as any no. of 0's

NOTE – Language is formed by appropriate strings & strings are formed by alphabets

Finite Automata (FA)

A finite automata is collection of 5-tuples

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

Q → Is a finite set of states, which is non empty

Σ → Is Input alphabet, indicates input set

q_0 → Is an initial state & q_0 is in Q

i.e. $q_0 \in Q$

F → Is a set of Final states

δ → Is a transition function or mapping function. Using this function next state can be determined

Finite State Machine (FSM)

- Finite automata is the finite state machine
- FA is represented as below

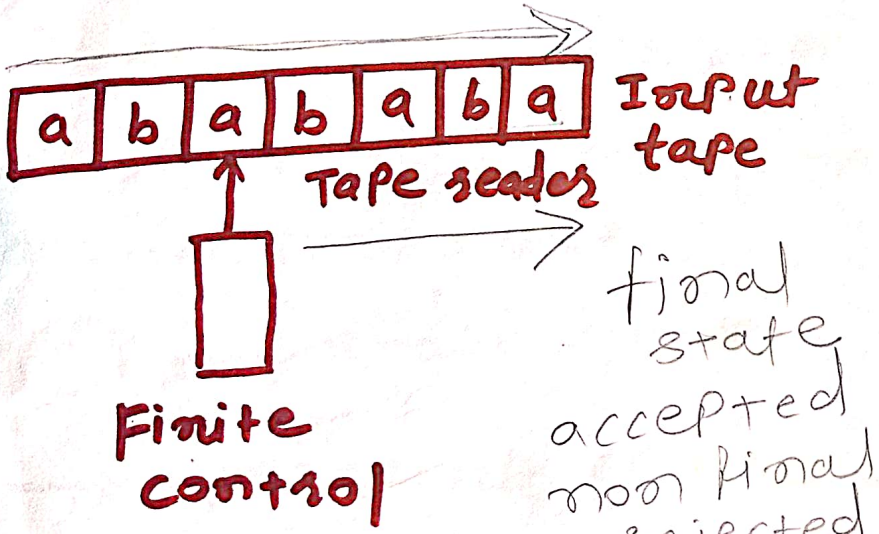


fig. Model of Finite Automata

- FA is represented using
  1) Input tape
  2) Tape reader
  3) Finite control

1) Input Tape -
- It is the linear tape having some no. of cells
- Each Input is placed in each cell

2) Tape Reader -
- Tape reader reads the cells one by one from left to right.
- At a time only one I/P symbol is read.

3) Finite control -
- The finite control decides the next state on receiving particular I/P from I/P tape

Representation of Finite Automata -

There are 2 notations for describing FA

① Transition Diagram ② Transition table

① Transtion Diagram

Notations used in Trans-tion diagram are

1) q_0 ⟹ state

2) ⟶ Represents transition from one state to anothes state

3) ⟶ q_1 ⟹ start state

4) q_f ⟹ Final state

Ex

② Transtion Table

I/P / Q	0	1
→ q_0	q_0	q_1
q_1*	q_1	q_0

5 - tuples

$Q \rightarrow \{q_0, q_1\}$

$\Sigma \rightarrow \{0, 1\}$

$q_0 \rightarrow q_0$

$F \rightarrow q_1$, $\delta \rightarrow$

Types of Finite Automata

Finite Automata

- Finite Automata without output
  - ① Deterministic finite Automata (DFA)
  - ② Non-Deterministic finite Automata (NFA)
  - ③ ε - NFA
- Finite Automata with output
  - ① Moore Machine
  - ② Mealy Machine

Deterministic Finite Automata (DFA)

- Deterministic means transition is deterministic.
- There is only one state to which an automata can transit from the current state on each Input.
- The word finite implies that the no. of states are Finite

Representation of DFA -

DFA is represented by 5-tuples

$M = (Q, \Sigma, \delta, q_0, F)$

$\delta(q_0, 0) \Rightarrow q_0$ q_2

$\Sigma = \{0, 1\}$

where $Q \rightarrow$ Is a finite set of states

$\Sigma \rightarrow$ Is Input alphabet, indicates

$\delta \rightarrow$ Is a transition function, is a function from $Q \times \Sigma$ to Q

$q_0 \rightarrow$ Is the initial state, $F \rightarrow$ set of final state

Designing of DFA -

Designing of DFA consist of following steps

1. **Step 1** - Theory
2. **Step 2** - Logic
3. **Step 3** - Implementation
   - State transition Diagram
   - State transition Table

   5-tuples
4. **Step 4** - Example
   - Acceptance of string
   - Rejection of string

Examples on counting of Symbols

① Design DFA which accepts the odd no. of 1's & any no. of 0's over the alphabet $\{0,1\}$.

$$L = \{010, 01\underline{0}1\underline{0}1, 0111, 01110 \ldots\}$$

Step 1 - Theory

Design DFA which accepts the odd no. of 1's & any no. of 0's over the alphabet $\{0,1\}$

Step 2 - Logic

No. of 1's are even $\rightarrow q_0$

No. of 1's are odd $\rightarrow q_1$

$\Sigma = \{0, 1\}$ $\quad \delta \rightarrow Q \times \Sigma \Rightarrow Q$

✓ **q_0** $q_0\ 0110 \xrightarrow{0} \underline{0^{1}1^{2}100}\ (q_0)$

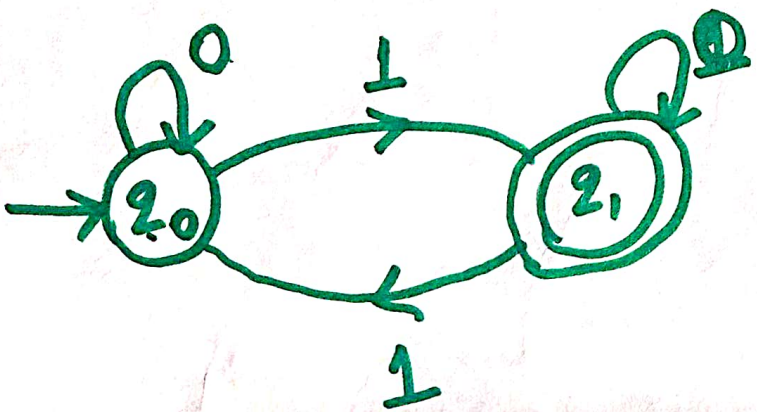
$q_0\ 0110 \xrightarrow{1} \underline{0^{1}1^{2}0^{3}1}\ (q_1)$

q_1 $q_1\ 010 \xrightarrow{0} \underline{0^{1}100}\ (q_1)$

$q_1\ 010 \xrightarrow{1} 0^{1}0^{2}1\ (q_0)$

Step 3 Implementation

State transition diagram



State transition table

Q \ Σ	0	1
→ q_0	q_0	q_1
q_1*	q_1	q_0

5-tuples

$M = \{Q, q_0, \delta, \Sigma, F\}$

$Q = \{q_0, q_1\}$

$q_0 = q_0$

$\delta(q_0, 0) \Rightarrow q_0$

$\Sigma = \{0, 1\}$

$F = q_1$

Step 4 - Example

$0110 \Rightarrow \delta(q_0, 0110)$

$\delta(q_0, 110)$

$\delta(q_1, 10)$

$\delta(q_0, 0)$

$\boxed{q_0}$ rejected

$010 \Rightarrow \delta(q_0, 010)$

$\delta(q_0, 10)$

$\delta(q_1, 0)$

$\boxed{q_1}$ accepted

② Design DFA accepting the following language over the alphabet $\{0,1\}$

(a) Number of 1's is even & Number of 0's is even.

(b) Number of 1's is odd & number of 0's is odd.

⇒ (a) No. of 1's is even & No. of 0's is even

Step 1 Theory

To design DFA which accepts even no. of 1's & 0's

Step 2 Logic

Situation		State
No. of 0's	No. of 1's	
even	even	→q_0*
even	odd	q_1
odd	even	q_2
odd	odd	q_3

$L(M) = \{1100, 1010, 0101, 0110 \cdots\}$

Transition (δ) $\quad \Sigma = \{0, 1\}$

q_0

$q_0\ 0011 \xrightarrow{0} 00110\ (q_2)$

$q_0\ 0011 \xrightarrow{1} 00111\ (q_1)$

---

q_1

$q_1\ 001 \xrightarrow{0} 0010\ (q_3)$

$q_1\ 001 \xrightarrow{1} 0011\ (q_0)$

---

q_2

$q_2\ 011 \xrightarrow{0} 0110\ (q_0)$

$q_2\ 011 \xrightarrow{1} 0111\ (q_3)$

---

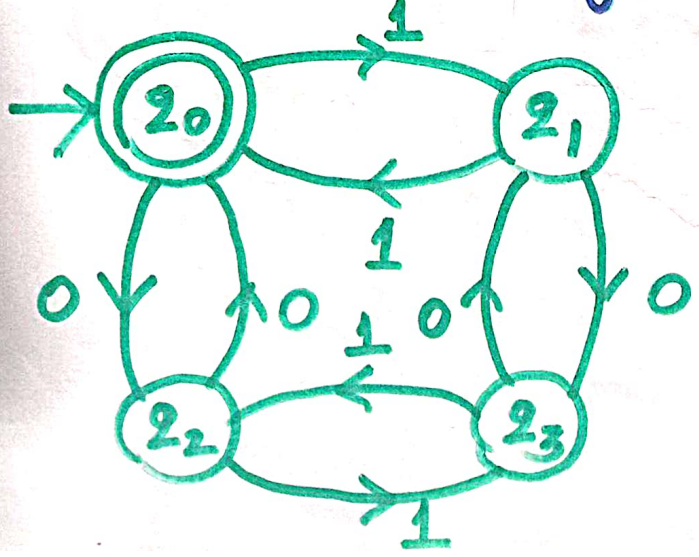
q_3

$q_3\ 01 \xrightarrow{0} 010\ (q_1)$

$q_3\ 01 \xrightarrow{1} 011\ (q_2)$

Step 3 Implementation

Transition Diagram



Transition Table

δ	0	1
$\rightarrow q_0^*$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	0

5-tuples

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

$F = q_0$

δ :

$\delta(q_0) \xrightarrow{0} q_2$

$\delta(q_0) \xrightarrow{1} q_1$

$\delta(q_1) \xrightarrow{0} q_3$

$\delta(q_1) \xrightarrow{1} q_0$

$\delta(q_2) \xrightarrow{0} q_0$

$\delta(q_2) \xrightarrow{1} q_3$

$\delta(q_3) \xrightarrow{0} q_1$

$\delta(q_3) \xrightarrow{1} q_2$

Step 4 Example

① eg. 0011

$(q_0, 0011) \Rightarrow \delta(q_0, 0011)$

$\delta(q_2, 011)$

$\delta(q_0, 11)$

$\delta(q_1, 1)$

$[q_0]$ final state

∴ I/p will be accepted by M/c

② eg. 001

$(q_0, 001) \Rightarrow \delta(q_0, 001)$

$\delta(q_2, 01)$

$\delta(q_0, 1)$

$[q_1]$ Non final state

∴ I/p will be rejected by M/c

③ Design DFA for the language over the alphabet $\{0,1\}$ which accepts all strings with exactly 2 one's

⇒ **Step 1** Theory

To design DFA which accepts all strings with exactly 2 one's

Step 2 Logic

String with 0 one's → $\rightarrow q_0$

1 one's → q_1

2 one's → q_2^*

> 2 one's → q_ϕ (dead state)

$L(M) = \{101, 110, 0110 \ldots\}$

Transition (δ) $\Sigma = \{0, 1\}$

q_0

$q_0\,00 \xrightarrow{0} 000\ (q_0)$

$q_0\,00 \xrightarrow{1} 001\ (q_1)$

---

q_1

$q_1\,01 \xrightarrow{0} 010\ (q_1)$

$q_1\,01 \xrightarrow{1} 011\ (q_2)$

---

q_2

$q_2\,011 \xrightarrow{0} 0110\ (q_2)$

$q_2\,011 \xrightarrow{1} 0111\ (q_\phi)$

---

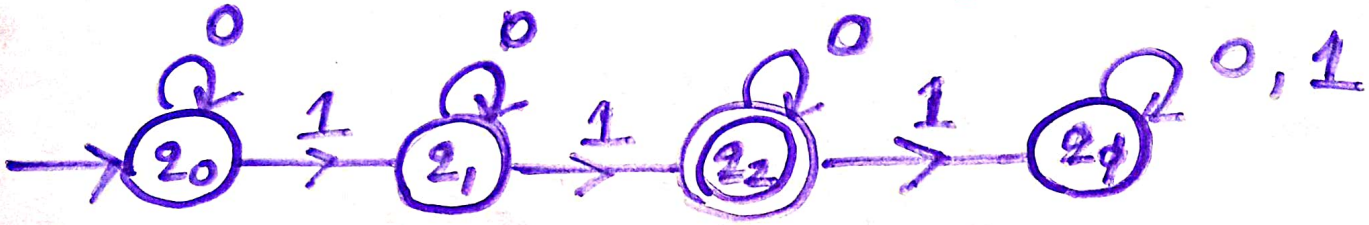
q_ϕ

$q_\phi\,0111 \xrightarrow{0} 01110\ (q_\phi)$

$q_\phi\,0111 \xrightarrow{1} 01111\ (q_\phi)$

Step 3 Implementation

Transition Diagram



Transtion Table

Q	0	1
→ q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_ϕ
q_ϕ	q_ϕ	q_ϕ

5-tuples

$M = (Q, q_0, \Sigma, \delta, F)$

$Q = \{q_0, q_1, q_2, q_\phi\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

$F = q_2$

δ : $\delta(q_0) \xrightarrow{0} q_0$

$\delta(q_0) \xrightarrow{1} q_1$

$\delta(q_1) \xrightarrow{0} q_1$

$\delta(q_1) \xrightarrow{1} q_2$

$\delta(q_2) \xrightarrow{0} q_2$

$\delta(q_2) \xrightarrow{1} q_\phi$

$\delta(q_\phi) \xrightarrow{0} q_\phi$

$\delta(q_\phi) \xrightarrow{1} q_\phi$

Step 4 Example

① eg. 011

$(q_0, 011) \Rightarrow \delta(q_0, 011)$

$\delta(q_0, 11)$

$\delta(q_1, 1)$

$\boxed{q_2}$ final state

∴ I/P will accept

② eg. 0111

$(q_0, 0111) \Rightarrow \delta(q_0, 0111)$

$\delta(q_0, 111)$

$\delta(q_1, 11)$

$\delta(q_2, 1)$

q_ϕ Non final state

∴ Ip string will be rejected by M/C

④ Design DFA for the language over {a, b} for all strings starting with abb.

⇒ **Step 1** - Theory

To design DFA for the language over {a, b} for all strings starting with abb

Step 2 - Logic

start state → q_0

string starting with a → q_1

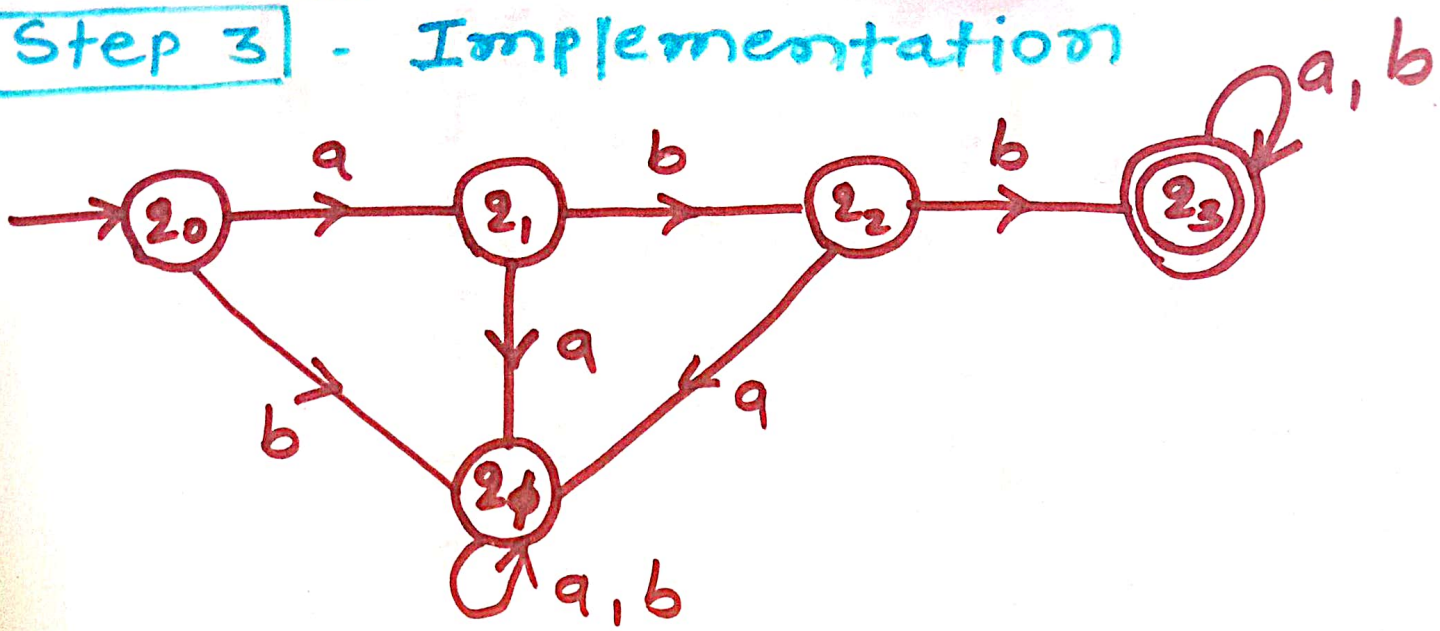
" ab → q_2

abb → q_3

failure state → q_ϕ

- First I/P as b will take the machine to failure state
- First 2 I/P's aa will take the machine to failure state.
- First 3 I/P's aba will take the machine to a failure state
- First 3 I/P's as abb will take the machine to a final state

Step 3 - Implementation



State transition Diagram

Q \ Σ	a	b
→ q_0	q_1	q_ϕ
q_1	q_ϕ	q_2
q_2	q_ϕ	q_3
q_3*	q_3	q_3
q_ϕ	q_ϕ	q_ϕ

State Transition Table

5-tuples

$M = (Q, q_0, \Sigma, \delta, F)$

$Q = \{q_0, q_1, q_2, q_3, q_\phi\}$

$q_0 = q_0$

$\Sigma = \{a, b\}$

⑤ Design DFA which accepts all strings with abb as substring.

⇒ **Step 1** - Theory

To design DFA which accepts all strings with abb as substring.

Step 2 - Logic

q_0 → It is the starting state & indicates that nothing of relevance to complete abb has been seen.

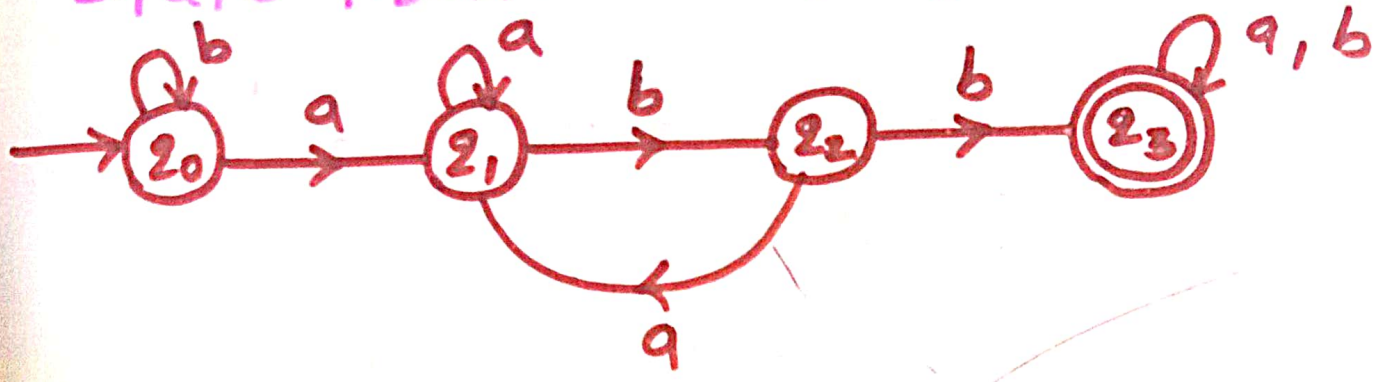
q_1 → Preceding character is a & bb is required to complete abb.

q_2 → Preceding characters are ab & b is required to complete abb.

q_3 → Preceding characters are abb & the substring abb has been seen.

Step 3 Implementation

State Transition Diagram



State Transition Table

Q \ Σ	a	b
→ q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3*	q_3	q_3

5 - tuples

$M = (Q, q_0, \Sigma, \delta, F)$

$Q = \{q_0, q_1, q_2, q_3\}$

$q_0 = q_0$

$\Sigma = \{a, b\}$

$F = q_3$

δ : $\delta(q_0, a) = q_1$

$\delta(q_0, b) = q_0$

$\delta(q_1, a) = q_1$

$\delta(q_1, b) = q_2$

$\delta(q_2, a) = q_1$

$\delta(q_2, b) = q_3$

$\delta(q_3, a) = q_3$

$\delta(q_3, b) = q_3$

Step 4 Example

Ex. babb

$(q_0, babb) \Rightarrow$

$\delta(q_0, babb)$

$\delta(q_0, abb)$

$\delta(q_1, bb)$

$\delta(q_2, b)$

q_3 Final state

$\therefore$ string is accepted

Ex. abab

$(q_0, abab) \Rightarrow$

$\delta(q_0, abab)$

$\delta(q_1, bab)$

$\delta(q_2, ab)$

$\delta(q_1, b)$

q_2 Non final state

$\therefore$ string
reje

Q) Design DFA which can accept a decimal no. divisible by 3 (residue mod 3).

⇒ **Step-1** Theory

To design DFA which accept a decimal no. divisible by 3.

Step-2 - Logic

A decimal system has 10 alphabets

$$\Sigma = \{ \frac{0}{3}, \frac{1}{3}, \frac{2}{3}, \frac{3}{3}, \frac{4}{3}, \frac{5}{3}, \frac{6}{3}, \frac{7}{3}, \frac{8}{3}, \frac{9}{3} \}$$

$$R = \{ \underline{0}, \underline{\underline{1}}, \underline{\underline{\underline{2}}}, \underline{0}, \underline{\underline{1}}, \underline{\underline{\underline{2}}}, \underline{0}, \underline{\underline{1}}, \underline{\underline{\underline{2}}}, \underline{0} \}$$

The running remainder could be

0 R → Associated state → q_0^*
1 R → Associated state, q_1
2 R → Associated state, q_2

Transition behaviour will depend on the current remainder (current state) & next I/P.

These 2 will be used to get the next remainder (next) state

Consider the transition for just 3 I/P's

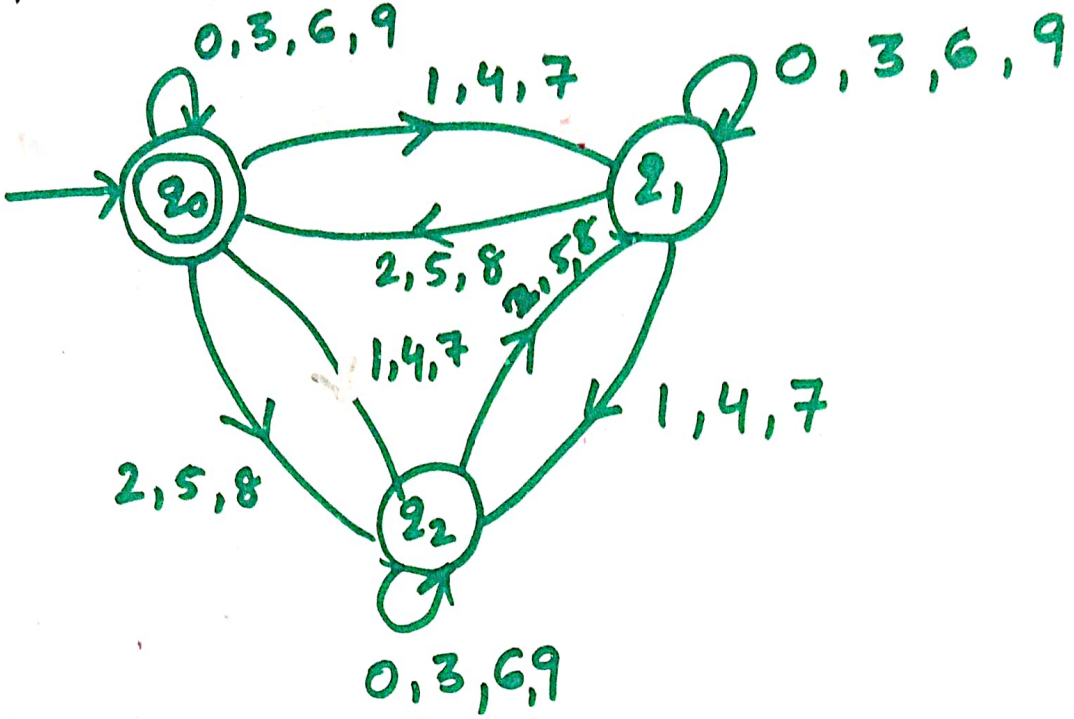
0 for (0, 3, 6, 9)
1 for (1, 4, 7)
2 for (2, 5, 8)

Transition behaviour is explained as below

Current remainder	Next I/P	calculation of remainder	Remainder
0	0	$00 \rightarrow 0 \text{ MOD } 3 \rightarrow 0$	q_0
	1	$01 \rightarrow 1 \text{ MOD } 3 \rightarrow 1$	q_1
	2	$02 \rightarrow 2 \text{ MOD } 3 \rightarrow 2$	q_2
1	0	$10 \rightarrow 10 \text{ MOD } 3 \rightarrow 1$	q_1
	1	$11 \rightarrow 11 \text{ MOD } 3 \rightarrow 2$	q_2
	2	$12 \rightarrow 12 \text{ MOD } 3 \rightarrow 0$	q_0
2	0	$20 \rightarrow 20 \text{ MOD } 3 \rightarrow 2$	q_2
	1	$21 \rightarrow 21 \text{ MOD } 3 \rightarrow 0$	q_0
	2	$22 \rightarrow 22 \text{ MOD } 3 \rightarrow 1$	

Step 3 Implementation

State transition diagram



State Transition Table

Q \ Σ	(0,3,6,9)	(1,4,7)	(2,5,8)
→ q_0*	q_0	q_1	q_2
q_1	q_1	q_2	q_0
q_2	q_2	q_0	q_1

5 - tuples

$M = (Q, q_0, \Sigma, \delta, F)$

$Q = \{q_0, q_1, q_2\}$

$q_0 = q_0$

$\Sigma = \{0,1,2,3,4,5,6,7,8,9\}$

$F = q_0$

Step 4 Example

① eg. 30

$(q_0, 30) \Rightarrow \delta(q_0, 30)$

$\delta(q_0, 0)$

$\boxed{q_0}$ final state

$\therefore$ No. is accepted by M/c

② eg - 25

$(q_0, 25) \Rightarrow \delta(q_0, 25)$

$\delta(q_2, 5)$

$\boxed{q_1}$ Non final state

$\therefore$ No. is rejected by M/c

Equivalence of DFA

Two finite automata M1 & M2 are said to be equivalent if they accept the same language.

i.e. $L(M1) = L(M2)$

Two finite automata M1 & M2 are not equivalent over Σ, if there exist a ω such that

$\omega \in L(M1)$ & $\omega \notin L(M2)$

or

$\omega \notin L(M1)$ & $\omega \in L(M2)$

i.e. one DFA reaches a final state on application of ω & other reaches a non final state

Show whether the automata M1 & M2 in given fig ⓐ & ⓑ are equivalent or not.

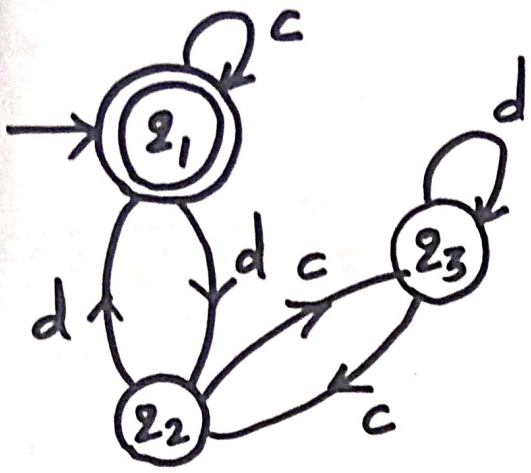


fig ⓐ M1

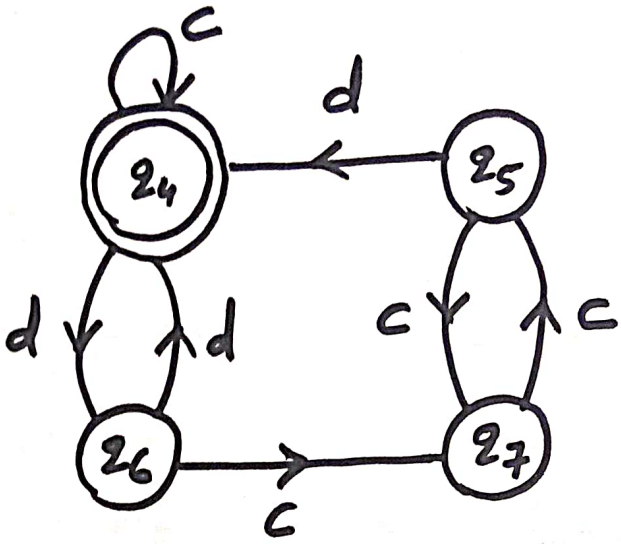


fig. ⓑ M2

⇒ construction of combined DFA M3

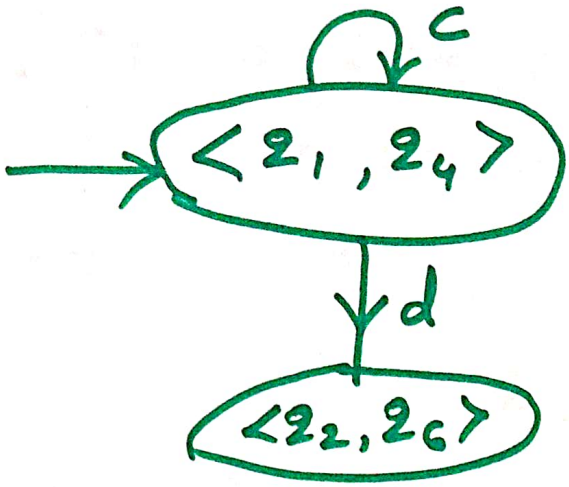
Step 1 – We can start with combined state $\langle q_1, q_4 \rangle$, where q_1 is start state of M1 & q_4 is the start state of M2

$\delta_3(\langle q_1, q_4 \rangle, c) \Rightarrow \langle \delta_1(q_1, c), \delta_2(q_4, c) \rangle$

$\Rightarrow \langle q_1, q_4 \rangle$

$\delta_3(\langle q_1, q_4 \rangle, d) \Rightarrow \langle \delta_1(q_1, d), \delta_2(q_4, d) \rangle$

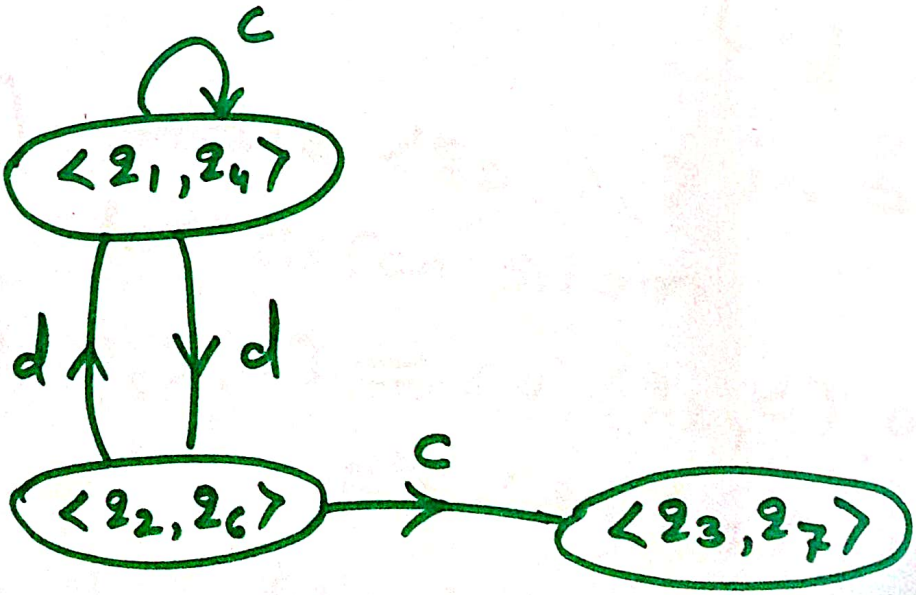
$\Rightarrow \langle q_2, q_6 \rangle$



Step 2 State $\langle q_2, q_6 \rangle$ is selected for expansion

$$\delta_3(\langle q_2, q_6 \rangle, c) \Rightarrow \langle \delta_1(q_2, c) \rangle, \delta_2(q_6, c) \rangle$$
$$\Rightarrow \langle q_3, q_7 \rangle$$

$$\delta_3(\langle q_2, q_6 \rangle, d) \Rightarrow \langle \delta_1(q_2, d), \delta_2(q_6, d) \rangle$$
$$\Rightarrow \langle q_1, q_4 \rangle$$



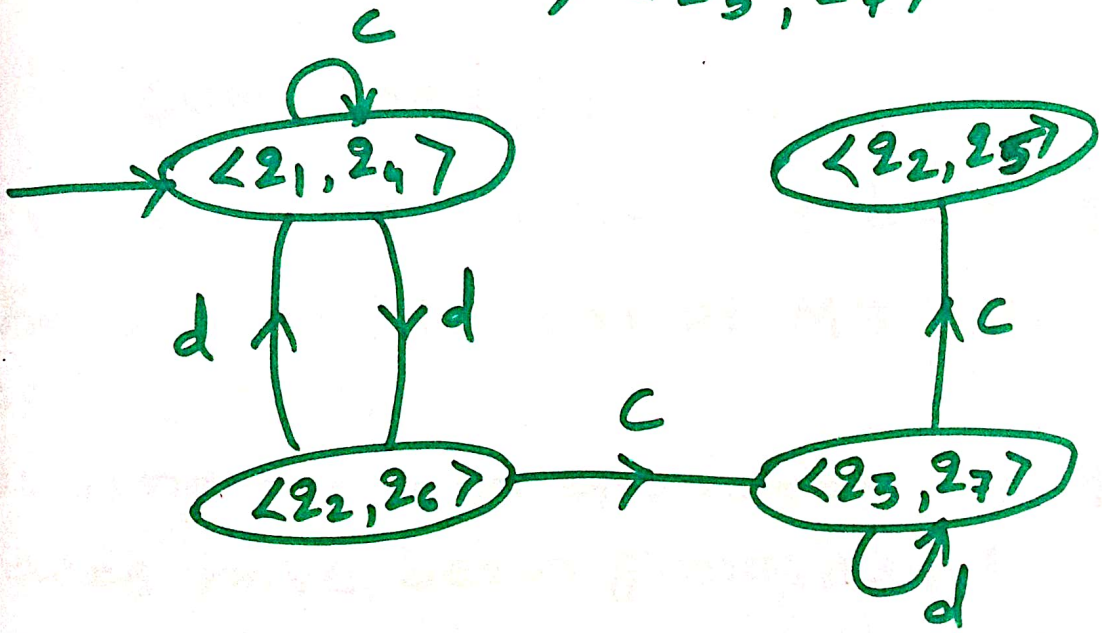
Step 3 - State $\langle q_3, q_7 \rangle$ is selected for expansion

$\delta_3(\langle q_3, q_7 \rangle, c) \Rightarrow \langle \delta_1(q_3, c), \delta_2(q_7, c) \rangle$

$\Rightarrow \langle q_2, q_5 \rangle$

$\delta_3(\langle q_3, q_7 \rangle, d) \Rightarrow \langle \delta_1(q_3, d), \delta_2(q_7, d) \rangle$

$\Rightarrow \langle q_3, q_7 \rangle$



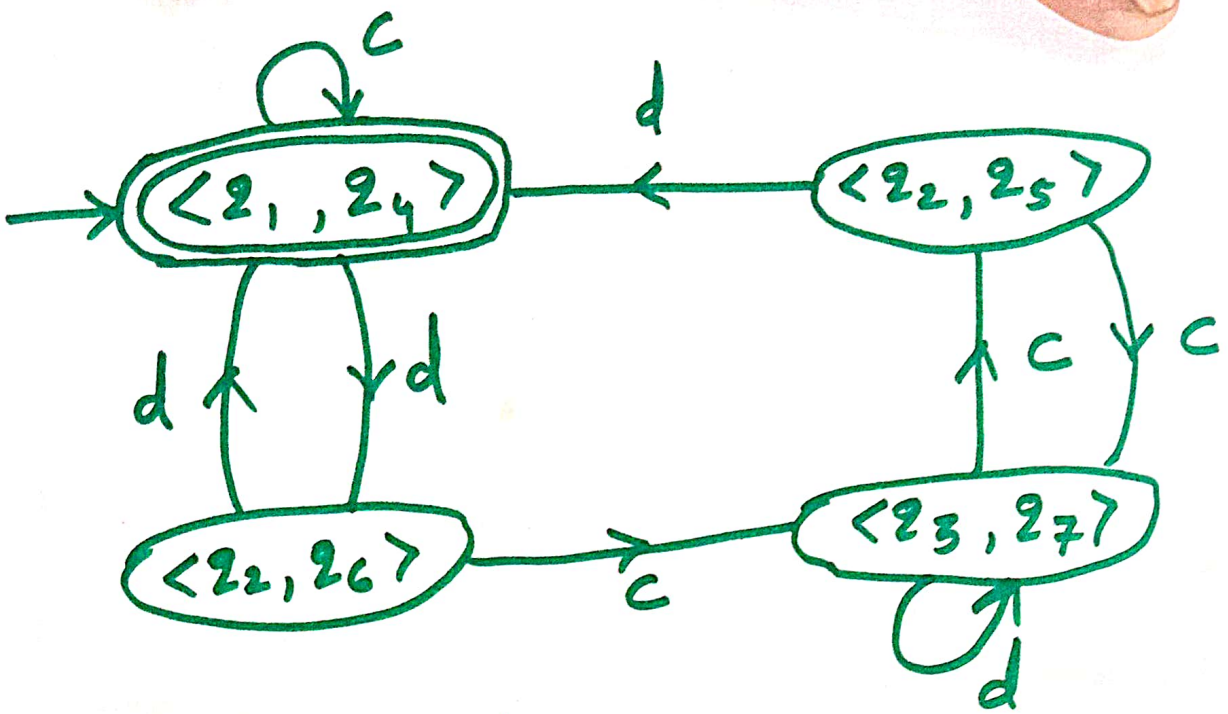
Step 4 - State $\langle q_2, q_5 \rangle$ is selected for expansion

$\delta_3(\langle q_2, q_5 \rangle, c) \Rightarrow \langle \delta_1(q_2, c), \delta_2(q_5, c) \rangle$

$\Rightarrow \langle q_3\, q_7 \rangle$

$\delta_3(\langle q_2, q_5 \rangle, d) \Rightarrow \langle \delta_1(q_2, d), \delta_2(q_5, d) \rangle$

$\Rightarrow \langle q_1, q_4 \rangle$



Combined DFA M3

- Every state in M3 has been expanded
- The construction of M3 i.e. combined DFA is over
- During the construction of M3 four states have been generated

① $\langle q_1, q_4 \rangle$: Both q_1 & q_4 are final states

② $\langle q_2, q_5 \rangle$: Both q_2 & q_5 are non final states

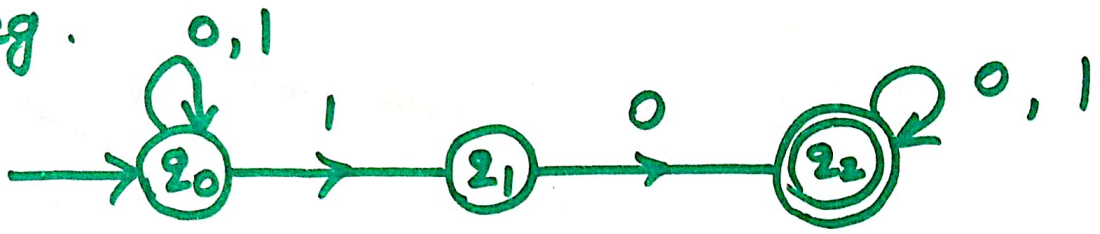
③ $\langle q_2, q_6 \rangle$: Both q_2 & q_6 are non final states

④ $\langle q_3, q_7 \rangle$: Both q_3 & q_7 are non final states

∴ We can say that M1 & M2 are equivalent

Non Deterministic finite Automata (NFA)

- NFA can reside in multiple states at the same time.
- eg.



- If the automata is in the state q_0 & the next I/P symbol is 1 then the next state will be either
  ① q_0 ② q_1
- Thus the transition of q_0 on I/P 1 can not be determined uniquely, such machines are called as NFA
- NFA can be in several states on same I/P at any time.
- Every NFA can be converted into an equivalent DFA.
- NFA can be designed with less no. of states as compared to DFA
  Due to non determinism, NFA takes more time to recognize a string

Representation of NFA -

NFA is represented by 5-tuples

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

$Q \rightarrow$ A finite set of states

$\Sigma \rightarrow$ A finite set of inputs

$\delta \rightarrow$ A transition function from $Q \times \Sigma$ to the power set of Q i.e. 2^Q

$q_0 \rightarrow$ Is the initial state, $q_0 \in Q$

$F \rightarrow$ Is the set of final states, $F \subseteq Q$

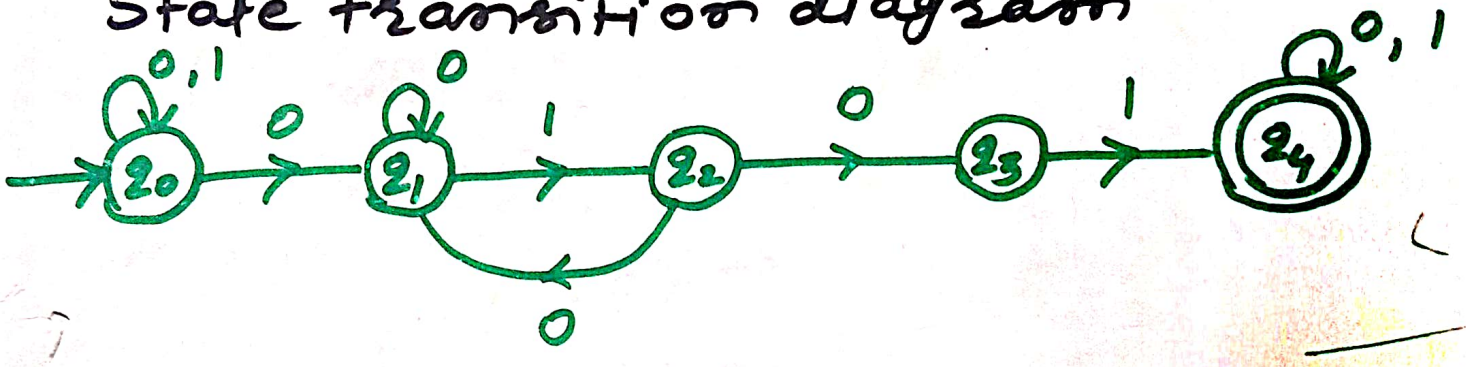
① Draw a NFA to accept strings containing the substring 0101

⇒ **Step 1** Theory

To design NFA to accept strings containing the substring 0101.

Step 2 - Implementation

State transition diagram



State transition table

q \ Σ	0	1
→q_0	$\{q_0, q_1\}$	q_0
q_1	q_1	q_2
q_2	$\{q_3, q_2\}$	ϕ
q_3	ϕ	q_4
q_4^*	q_4	q_4

5. Tuples

$M = (Q, q_0, \Sigma, \delta, F)$

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$q_0 = q_0$

$\Sigma = \{0, 1\}$

$F = q_4$

δ :

$\delta(q_0, 0) \rightarrow \{q_0, q_1\}$

$\delta(q_0, 1) \rightarrow q_0$

$\delta(q_1, 0) \rightarrow q_1$

$\delta(q_1, 1) \rightarrow q_2$

$\delta(q_2, 0) \rightarrow \{q_3, q_2\}$

$\delta(q_2, 1) \rightarrow \phi$

$\delta(q_3, 0) \rightarrow \phi$

$\delta(q_4, 0) \rightarrow q_4$

$\delta(q_4, 1) \rightarrow q_4$

Step 3 Example

eg. 101010

$(q_0, 101010) \Rightarrow \delta(q_0, 101010)$

$\delta(q_0, 01010)$

$\delta(q_1, 1010)$

$\delta(q_2, 010)$

$\delta(q_3, 10)$

$\delta(q_4, 0)$

$\boxed{q_4}$ final state

$\therefore$ String is accepted

② Construct NFA that accepts any positive no. of occurences of various strings from the following language L given by

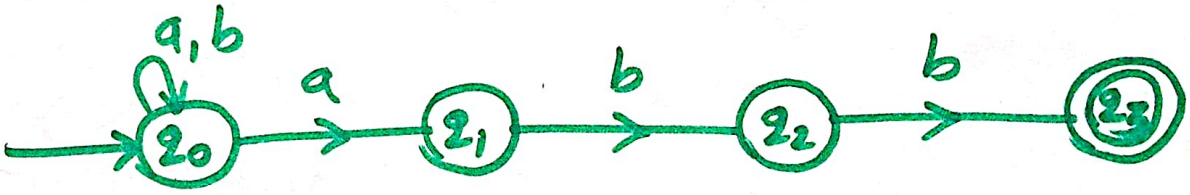
$L = \{x \in (a,b) \mid x \text{ ends with } abb\}$

⇒ **Step 1** Theory

To construct NFA in which all strings ends with abb

Step 2 Implementation

State transition diagram



State transition table

q / Σ	a	b
→ q_0	$\{q_0, q_1\}$	q_0
q_1	ϕ	q_2
q_2	ϕ	q_3
q_3*	ϕ	ϕ

5 - Tuples

$$M = (Q, \Sigma_0, \Sigma, \delta, F)$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma_0 = q_0$$

$$\Sigma = \{a, b\}$$

$$F = q_3$$

δ :

$$\delta(q_0, a) \rightarrow \{q_0, q_1\}$$
$$\delta(q_0, b) \rightarrow q_0$$
$$\delta(q_1, a) \rightarrow \phi$$
$$\delta(q_1, b) \rightarrow q_2$$
$$\delta(q_2, a) \rightarrow \phi$$
$$\delta(q_2, b) \rightarrow q_3$$
$$\delta(q_3, a) \rightarrow \phi$$
$$\delta(q_3, b) \rightarrow \phi$$

Step 3 Example

eg- aabb , $(q_0, aabb) \Rightarrow \delta(q_0, aabb)$

$\delta(q_0, abb)$

$\delta(q_1, bb)$

$\delta(q_2, b)$

$[q_3]$ final state

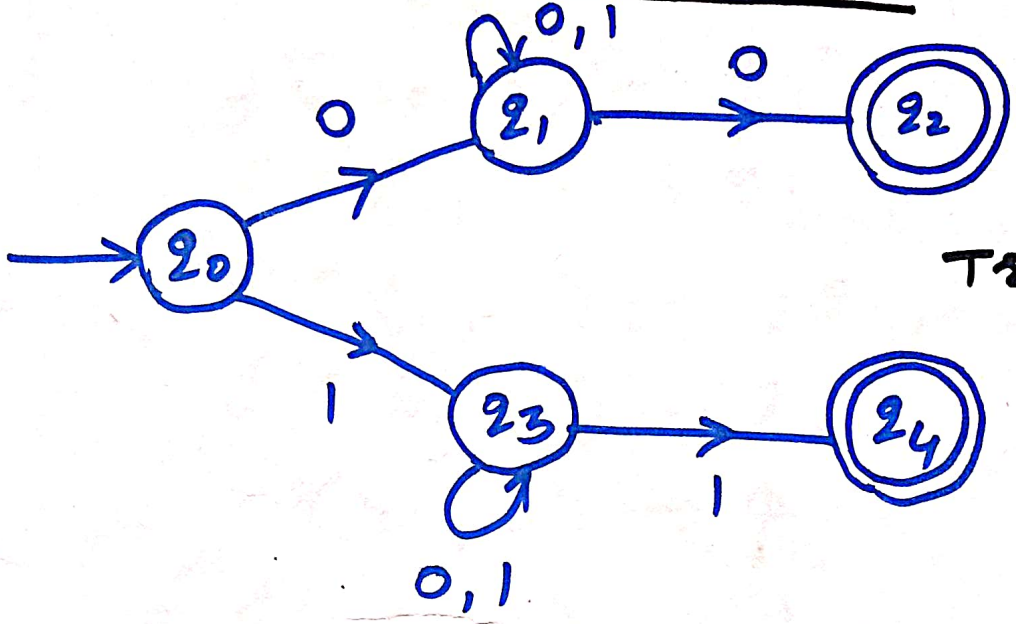
$\therefore$ string is accepted

③ Design NFA for binary no. where the first & last digits are same

⇒ **Step 1** **Theory**

To design NFA for binary no. where the first & last digits are same

Step 2 **Implementation**



State Transition diagram

State Transition table

Q	ε	0	1
→ q_0		q_1	q_3
q_1		$\{q_1, q_2\}$	q_1
q_2		ϕ	ϕ
q_3		q_3	$\{q_3, q_4\}$
q_4*		ϕ	ϕ

5-tuples

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$q_0 = q_0$

$\Sigma = \{0, 1\}$

$F = \{q_2, q_4\}$

δ :

$\delta(q_0, 0) \rightarrow q_1$

$\delta(q_0, 1) \rightarrow q_3$

$\delta(q_1, 0) \rightarrow \{q_1, q_2\}$

$\delta(q_1, 1) \rightarrow q_1$

$\delta(q_2, 0) \rightarrow \phi$

$\delta(q_2, 1) \rightarrow \phi$

$\delta(q_3, 0) \rightarrow q_3$

$\delta(q_3, 1) \rightarrow \{q_3, q_4\}$

$\delta(q_4, 0) \rightarrow \phi$

$\delta(q_4, 0) \rightarrow \phi$

Step 3| Example

eg. 010

$(q_0, 010) \Rightarrow \delta(q_0, 010)$

$\delta(q_1, 10)$

$\delta(q_1, 0)$

q_2 final state

$\therefore$ String is accepted

eg. 101

$(q_0, 101) \Rightarrow \delta(q_0, 101)$

$\delta(q_3, 01)$

$\delta(q_3, 1)$

q_4 final state

$\therefore$ String is accepted

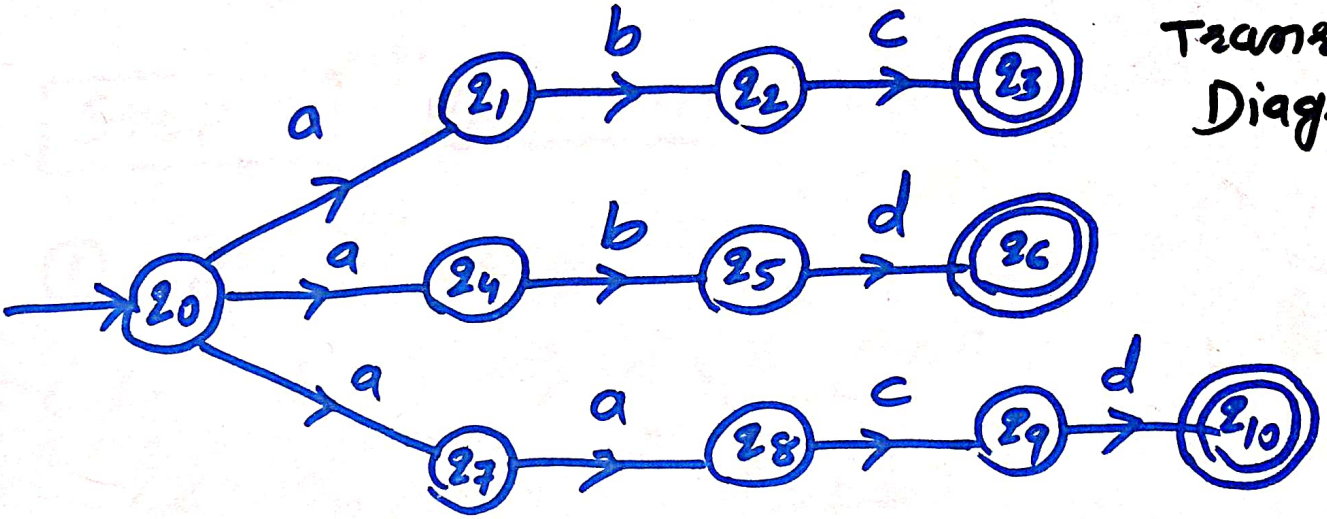
④ Design a NFA to recognize the following sets of strings : abc, abd, aacd. Assume that alphabet is {a, b, c, d}.

⇒ **Step 1** Theory

To design NFA to recognize the following sets of strings : abc, abd, aacd.

Step 2 Implementation

State Transition Diagram



Q	ε	a	b	c	d
→ q0		{q1, q4, q7}	ϕ	ϕ	ϕ
q1		ϕ	q2	ϕ	ϕ
q2		ϕ	ϕ	q3	ϕ
q3*		ϕ	ϕ	ϕ	ϕ
q4		ϕ	q5	ϕ	ϕ
q5		ϕ	ϕ	ϕ	q6
q6*		ϕ	ϕ	ϕ	ϕ
q7		q8	ϕ	ϕ	ϕ
q8		ϕ	ϕ	q9	ϕ
q9		ϕ	ϕ	ϕ	q10
q10*		ϕ	ϕ	ϕ	ϕ

State Transition Table

5-tuples

$M = (Q, \Sigma, q_0, \delta, F)$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}\}$

$\Sigma = \{a, b, c, d\}$

$q_0 = q_0$

$F = \{q_5, q_6, q_{10}\}$

δ :

Step 3 Example

① eg. abc

$(q_0, abc) \Rightarrow \delta(q_0, abc)$

$\delta(q_1, bc)$

$\delta(q_2, c)$

q_5 final state

∴ string is accepted

$(q_0, abd) \Rightarrow \delta(q_0, abd)$

$\delta(q_4, bd)$

$\delta(q_5, d)$

q_6 final state

∴ string is accepted

③ eg. aacd

$\delta(q_0, aacd) = \delta(q_0, aacd)$

$\delta(q_7, acd)$

$\delta(q_8, cd)$

$\delta(q_9, d)$

$\boxed{q_{10}}$ final state

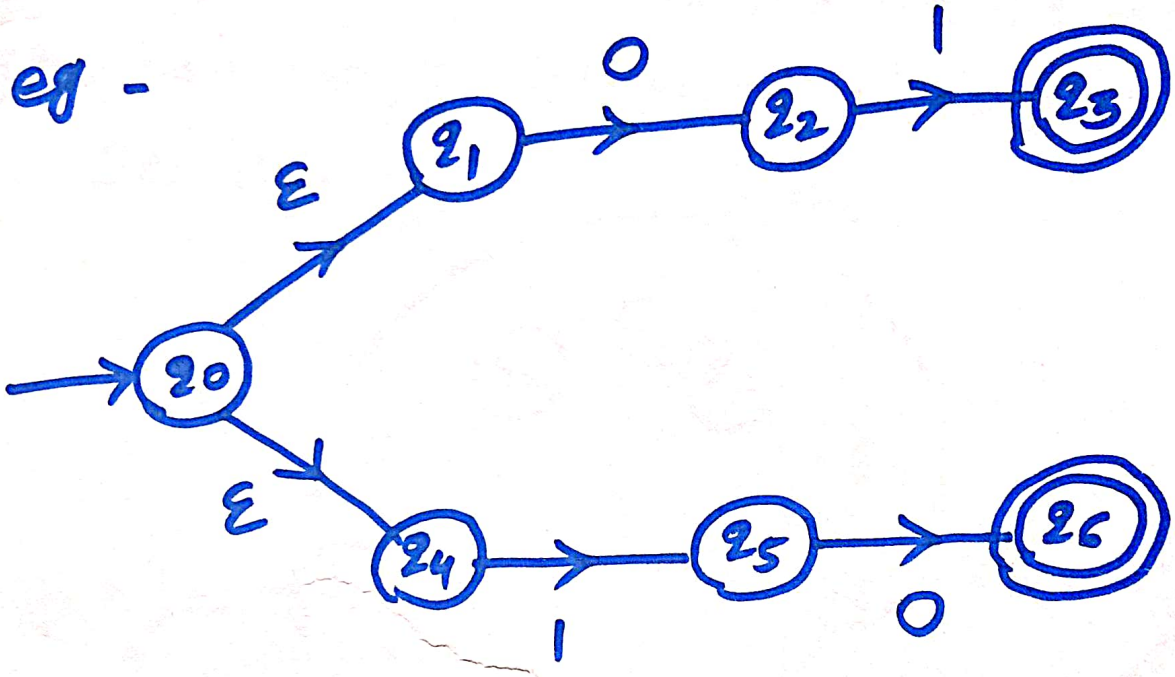
$\therefore$ String is accepted

NFA with ε - transition

- ε stands for null symbol / empty string.
- ε - transitions in NFA are given in order to move from one state to another state without having any symbol from I/P set Σ i.e. machine make a transition without any I/P
- ε transitions are used simply to change one state to another state
- Sometimes to reach final state we do not get proper state from start state, in such case we simply want to reach to certain state which leads to final state
- Such a transition to that specific state should be without any I/P symbol

- Hence we require some ε-moves by which a proper state can be obtained for reaching to the final state
- Thus ε-transitions plays an important role in NFA

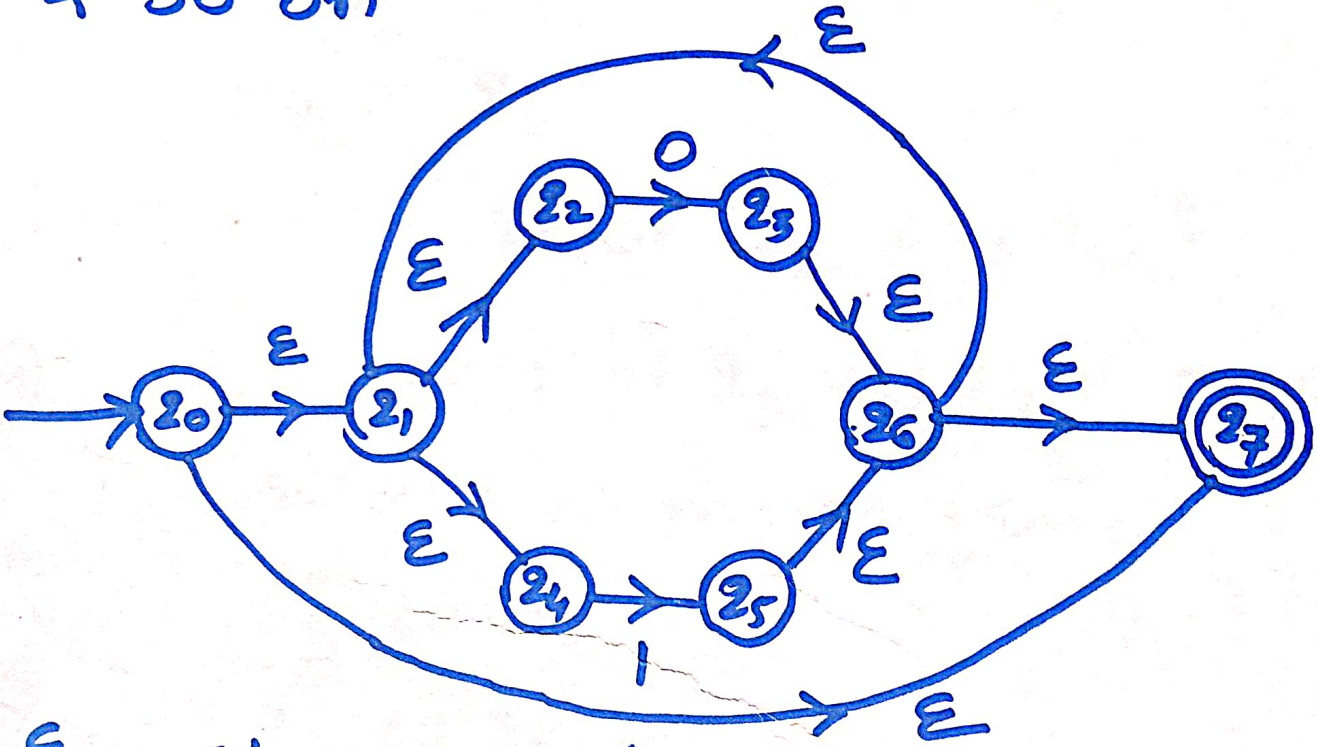
eg -



(Strings ending in 01 or 10)

ε - closure

- ε - closure of a state q_i includes q_i (because every state is at distance zero from itself)
- set of states reachable from q_i on ε - move
- set of states reachable from existing states in ε - closure using ε - move & so on



ε - closure of various states are as follows

① ε - closure of $q_0 = \{q_0, q_1, q_2, q_4, q_7\}$

There are ε - moves from

q_0 to q_1

q_1 to q_2

q_2 to q_4

q_0 to q_7

② ε - closure of $q_1 = \{q_1, q_2, q_4\}$

There are ε - moves from

q_1 to q_2

q_2 to q_4

③ ε - closure of $q_2 = \{q_2\}$

④ ε - closure of $q_3 = \{q_3, q_6, q_7, q_1, q_2, q_4\}$

There are ε - moves from

q_3 to q_6

q_6 to q_7

q_6 to q_1

q_1 to q_2

q_1 to q_4

⑤ ε - closure of $q_4 = \{q_4\}$

⑥ ε - closure of q_5 –

$q_5 = \{q_5, q_6, q_7, q_1, q_2, q_4\}$

There are ε - moves from

q_5 to q_6

q_6 to q_7

q_6 to q_1

q_1 to q_2

q_1 to q_4

⑦ ε - closure of $q_6 = \{q_6, q_7, q_1, q_2, q_4\}$

There are ε - moves from

q_6 to q_7

q_6 to q_1

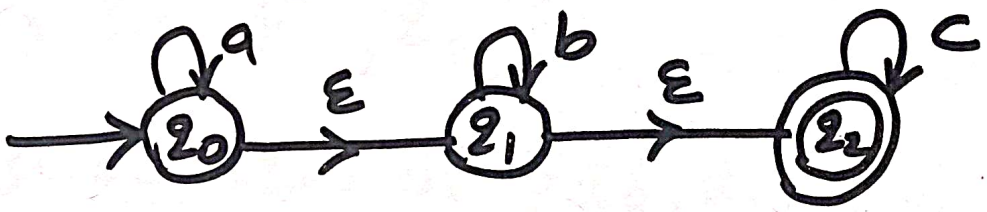
q_1 to q_2

q_1 to q_4

ε - NFA to DFA

- ε - NFA to DFA conversion is based on subset construction
- ε - closure is calculated for every state in the subset i.e. subset is extended by ε - closure of every state in the subset

① Find an equivalent DFA for the ε - NFA given in fig.



⇒ **Step 1** ε - closure of states is as follows

State	ε-closure
q_0	$\{q_0, q_1, q_2\}$
q_1	$\{q_1, q_2\}$
q_2	$\{q_2\}$

Step 2 ε - closure of initial state q_0 is taken as the first subset

ε - closure $(q_0) = \{q_0, q_1, q_2\}$

∴ a successor of $\{q_0, q_1, q_2\}$ =

ε - closure $[\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)]$

ε - closure $[q_0 \cup \phi \cup \phi]$

ε - closure $[q_0]$

$\{q_0, q_1, q_2\}$

∴ b successor of $\{q_0, q_1, q_2\}$ =

ε - closure $[\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)]$

ε - closure $[\phi \cup q_1 \cup \phi]$

ε - closure $[q_1]$

$\{q_1, q_2\}$

∴ b successor of $\{q_1, q_2\}$ =

ε-closure $[\delta(q_1, b) \cup \delta(q_2, b)]$

ε-closure $[q_1 \cup \phi]$

ε-closure $[q_1]$

$\{q_1, q_2\}$

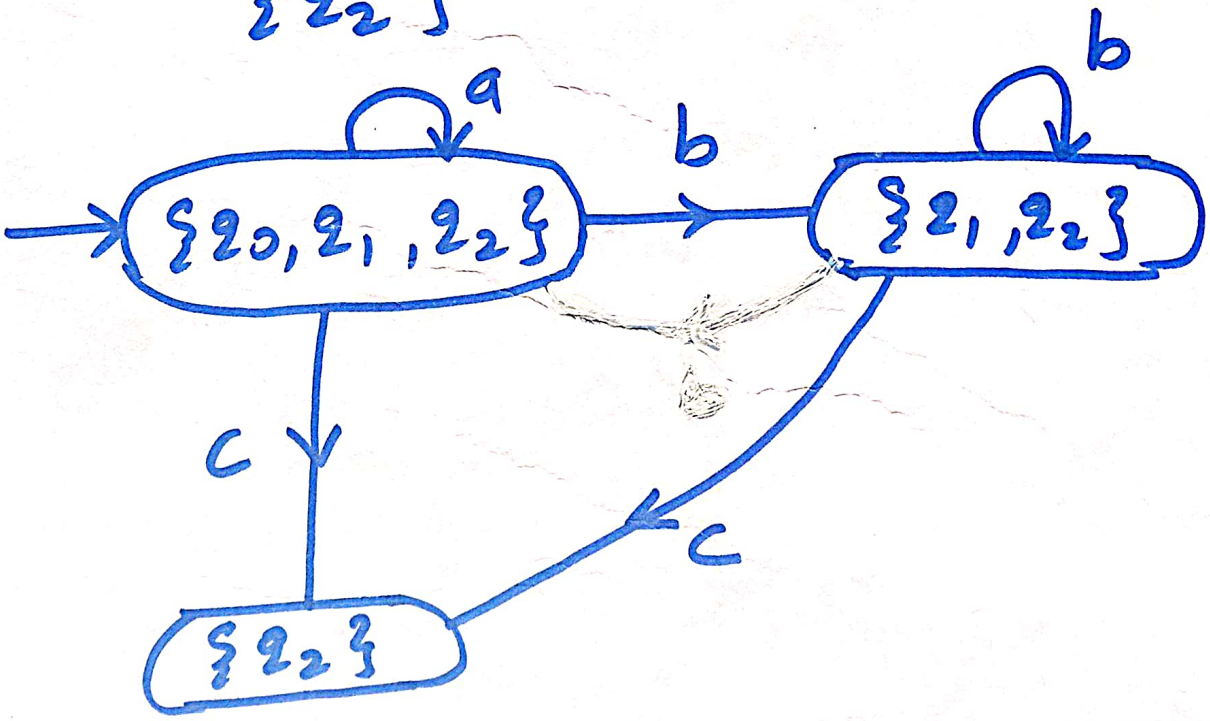
∴ c-successor of $\{q_1, q_2\}$ =

ε-closure $[\delta(q_1, c) \cup \delta(q_2, c)]$

ε-closure $[\phi \cup q_2]$

ε-closure $[q_2]$

$\{q_2\}$



Step 3 New subset $\{2_2\}$ is selected for subset construction

$\therefore$ a successor of $\{2_2\}$ =

ε-closure $[\delta(2_2, a)]$

ε-closure $[\phi]$

ϕ

$\therefore$ b successor of $\{2_2\}$ =

ε-closure $[\delta(2_2, b)]$

ε-closure $[\phi]$

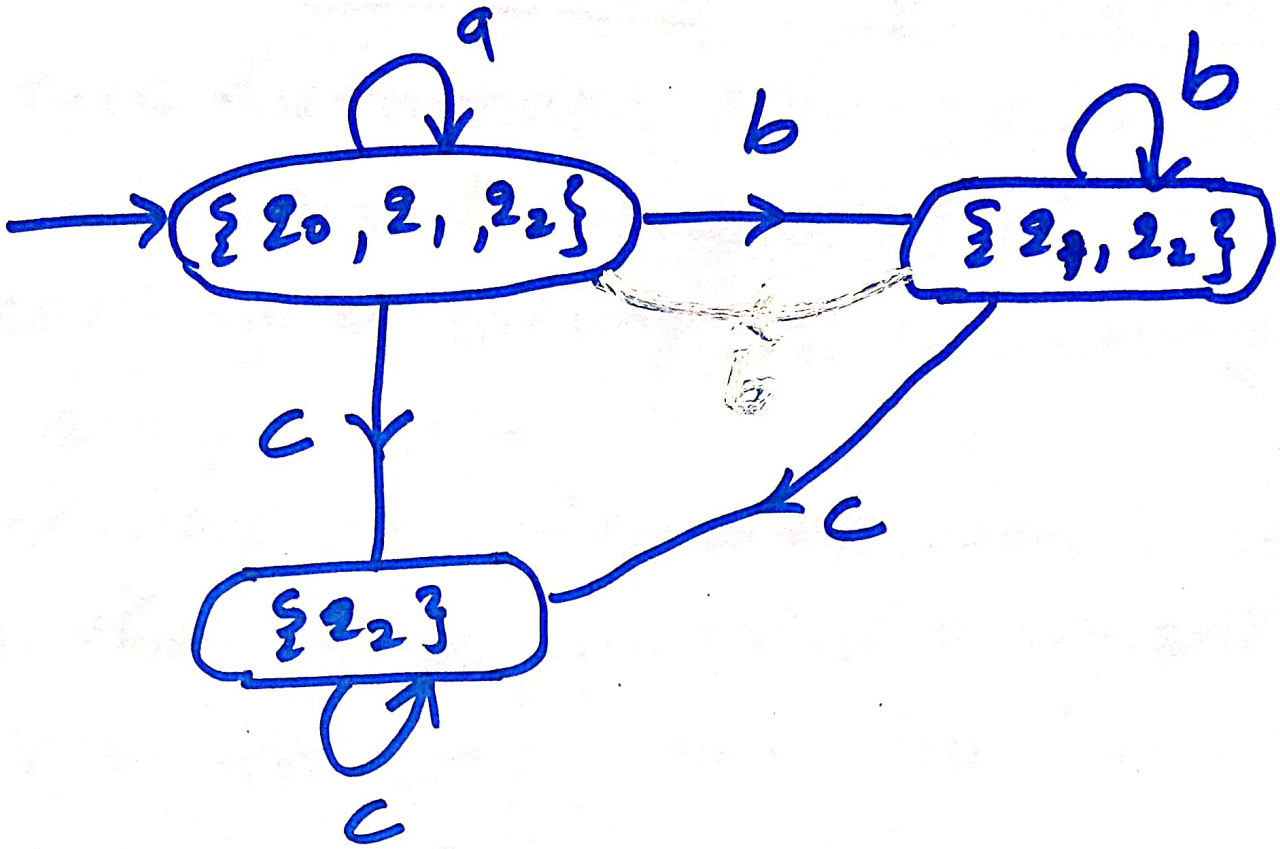
ϕ

$\therefore$ c successor of $\{2_2\}$ =

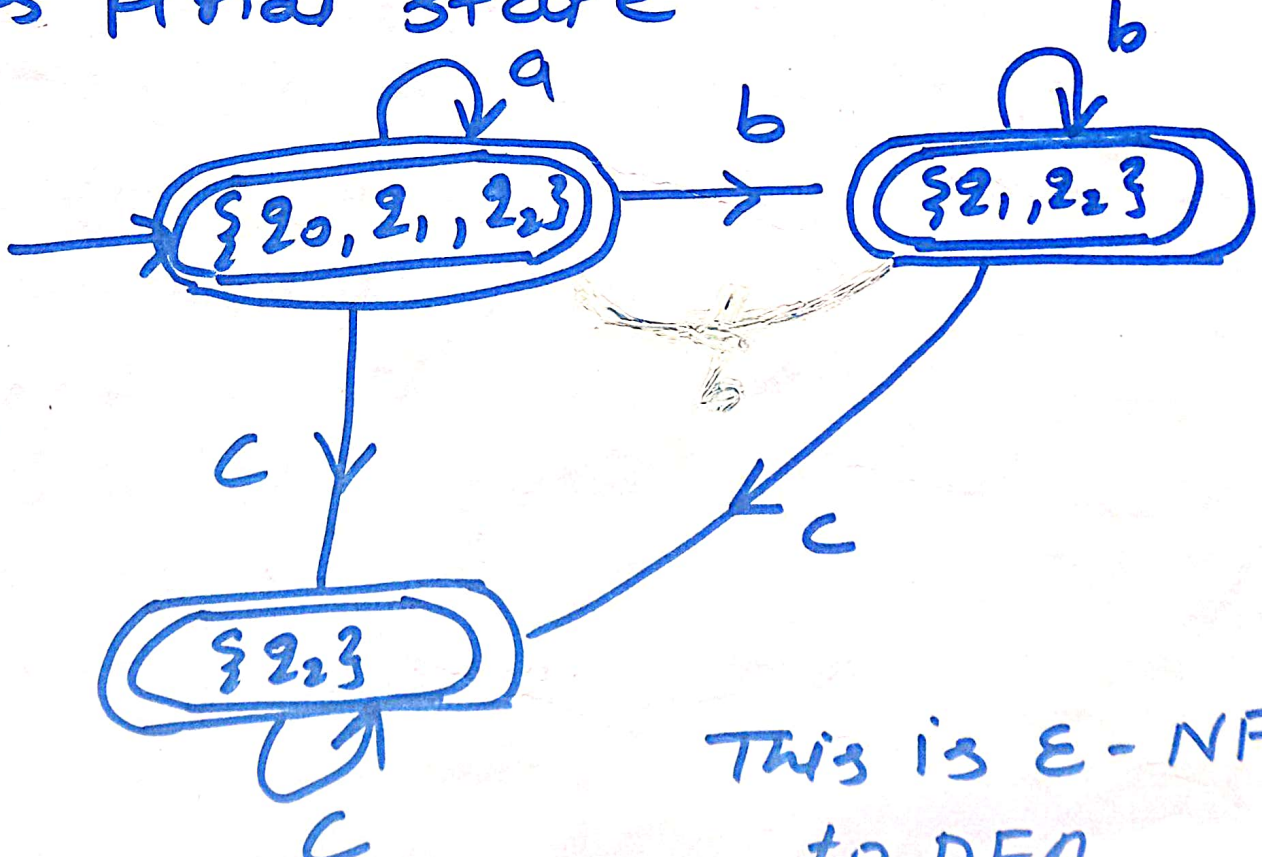
ε-closure $[\delta(2_2, c)]$

ε-closure $[2_2]$

$\{2_2\}$



how to show final state, so wherever q_2 is there make that as final state



This is ε-NFA to DFA

Finite Automata with output

- Finite automata like DFA & NFA have limited capability of either accepting a string or rejecting a string
- Acceptance of string was based on the reachability of a machine from starting state to final state
- FA can also be used as an output device & such machine do not have final states.
- Machine generate an output on every I/P & the value of the o/P is a function of current state & current I/P
- such machines are characterised by 2 behaviours

① state transition function (δ) (STF)
② output function (λ) (MAF)

There are 2 types of automata with o/p's
① Moore Machine ② Mealy Machine

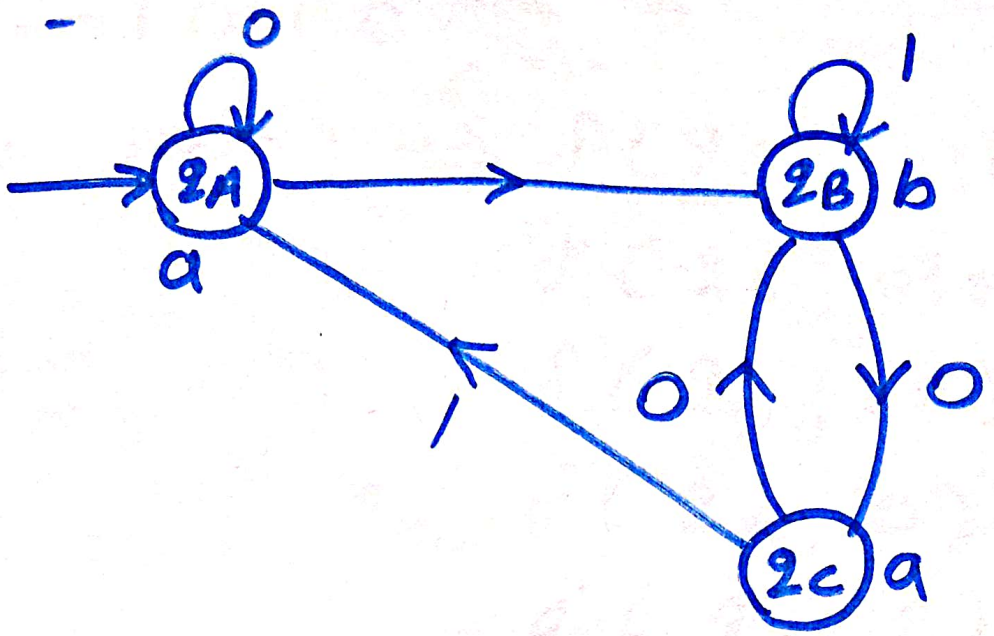
① Moore Machine

- It is the m/c with finite no. of states & for which the o/p symbol at a given time depends on the present state of the m/c.
- In moore m/c, an o/p symbol is associated with each state
- In moore m/c, an o/p symbol is associated with each state.
- In moore m/c, if the length of the I/P sequence is 'n', then the length of o/p sequence is 'n+1'
- It is represented by 6-tuples

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where, $Q \rightarrow$ Finite set of states
$\Sigma \rightarrow$ Finite I/P alphabet
$\Delta \rightarrow$ Finite o/p alphabet
$\delta \rightarrow$ state Function $\delta : Q \times \Sigma \rightarrow Q$
$\lambda \rightarrow$ Machine function $\lambda : Q \rightarrow \Delta$
$q_0 \rightarrow$ Initial state, $q_0 \in Q$

eg -



6 - tuples are

$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$

$Q = \{q_A, q_B, q_c\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

$\Delta = \{a, b\}$

δ :

Q \ Σ	0	1
→q_A	q_A	q_B
q_B	q_c	q_B
q_c	q_B	q_A

λ :

$\lambda(q_A) = a$

$\lambda(q_B) = b$

$\lambda(q_c) = a$

eg - 1 0 1 0

$(q_A, 1010) \Rightarrow \delta(q_A, 1010)$ a

$\delta(q_B, 010)$ b

$\delta(q_C, 10)$ a

$\delta(q_A, 0)$ a

$\delta(q_A, \varepsilon)$ a

I/P string length is 4

output string length is 5

① Design moore m/c to o/p EVEN or ODD depending on the no. of 1's over $\Sigma = \{0, 1\}$.

⇒ **Step 1** Theory

To design moore M/c to o/p EVEN or ODD

Step 2 Logic

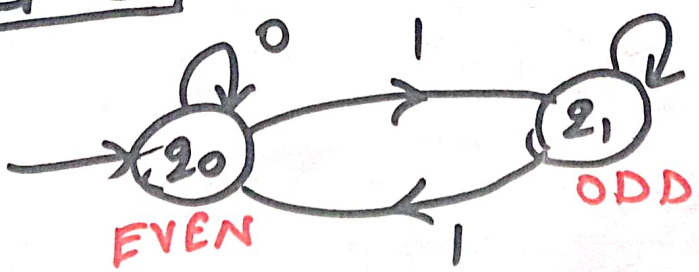
Considers the following states

$q_0 \rightarrow$ No. of 1's are even

$q_1 \rightarrow$ No. of 1's are odd

q \ Σ	0	1
→ q_0	11 → 110 (q_0)	11 → 111 (q_1)
q_1	10 → 100 (q_1)	10 → 101 (q_0)

Step 3 Implementation



δ :

Q \ Σ	0	1
→q_0	q_0	q_1
q_1	q_1	q_0

λ :

	output (λ)
→q_0	EVEN
q_1	ODD

6 - tuples

$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$

$\Delta = \{EVEN, ODD\}$

$q_0 = q_0$

Step 4 Example

eg. 101

$\delta(q_0, 101) \rightarrow$ $\delta(\underline{q_0}, 101)$ EVEN

$\delta(\underline{q_1}, 01)$ ODD

$\delta(\underline{q_1}, 1)$ ODD

$\delta(q_0, \varepsilon)$ EVEN

② Construct moore machine to o/p the remainder when binary no. is divided by 3 OR

Design Moore MIC to determine residue mod 3 for binary no

⇒ Theory

To design moore mlc to o/p the remainder when binary no. is divided by 3

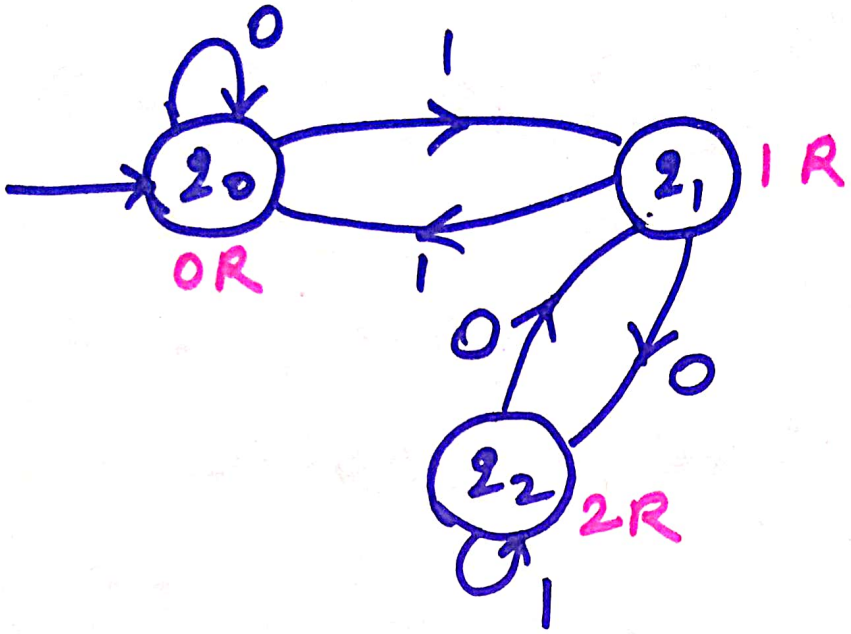
step 2 Logic

→ $q_0 \rightarrow 0$ Remainder state
$q_1 \rightarrow 1$ "
$q_2 \rightarrow 2$ "

	q \ Σ	0	1
OR	$\rightarrow q_0$	00 %3 = 0 (q_0)	01 %3 = 1 (q_1)
1R	q_1	10 = 2%3=2 (q_2)	11 = 3%3 = 0 (q_0)
2R	q_2	100=4%3=1 (q_1)	101 = 5%3 = 2 (q_2)

Step 3 Implementation

State transition diagram



State transition table

δ :

a \ Σ	0	1
→ q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

λ :

	output
→ q_0	0R
q_1	1R
q_2	2R

6 tuples

$M = (Q, z_0, \Sigma, \delta, \Delta, \lambda)$

$Q = \{z_0, z_1, z_2\}$

$z_0 = z_0$

$\Sigma = \{0, 1\}$

$\Delta = \{0R, 1R, 2R\}$

Step 4 Example

eg - 1011

$(z_0, 1011) \Rightarrow \delta(z_0, 1011)\ 0R$

$\delta(z_1, 011)\ 1R$

$\delta(z_2, 11)\ 2R$

$\delta(z_2, 1)\ 2R$

$\delta(z_2, \varepsilon)\ 2R$

$1011 = 11 \% 3 = 2 \quad (2R)$

③ Design moore mlc to o/p

A = If I/P ends in 101

B = If I/P ends in 110

C = otherwise o/p over $\Sigma = \{0, 1\}$

⇒ **Step 1** Theory

To design moore mlc to o/p

A = If I/P ends in 101

B = If I/P ends in 110

C = otherwise o/p over $\Sigma = \{0, 1\}$

Step 2 Logic

→ q_0 → I/P ends in 0

q_1 → I/P ends in 1

q_2 → I/P ends in 10

q_3 → I/P ends in 101

q_4 → I/P ends in 11

q_5 → I/P ends in 110

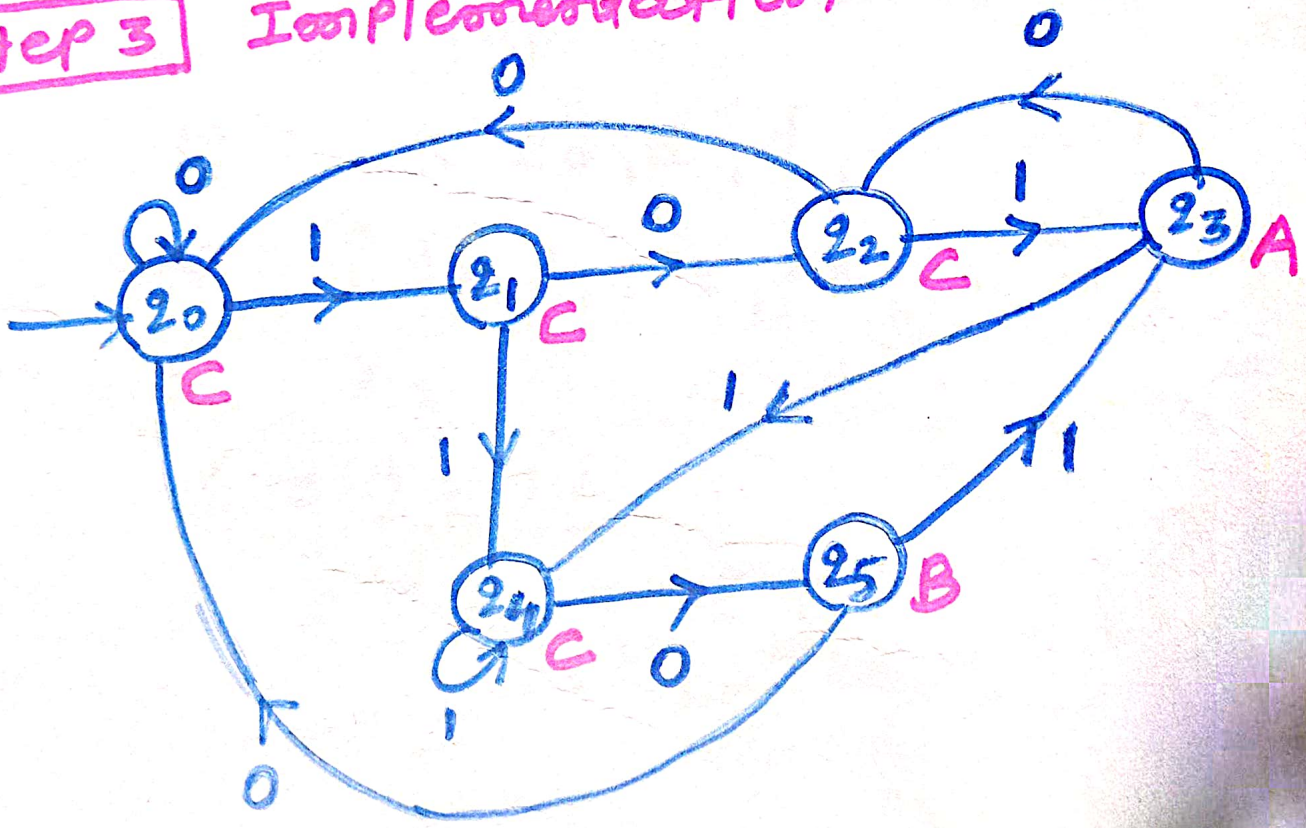
δ :

Q \ Σ	0	1
$0 \to q_0$	00 (q_0)	01 (q_1)
1 q_1	10 (q_2)	11 (q_4)
10 q_2	100 (q_0)	101 (q_3)
101 q_3	1010 (q_2)	1011 (q_4)
11 q_4	110 (q_5)	111 (q_4)
110 q_5	1100 (q_0)	1101 (q_3)

λ :

	output(λ)
q_0	C
q_1	C
q_2	C
q_3	A
q_4	C
q_5	B

Step 3 Implementation



State Transition Diagram

δ : State Transition Table

q \ Σ	0	1
→ q_0	q_0	q_1
q_1	q_2	q_4
q_2	q_0	q_3
q_3	q_2	q_4
q_4	q_5	q_4
q_5	q_0	q_3

λ :

	output(λ)
q_0	C
q_1	C
q_2	C
q_3	A
q_4	C
q_5	B

6 - tuples

$M = (Q, q_0, \Sigma, \delta, \Delta, \lambda)$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$q_0 = q_0$

$\Sigma = \{0, 1\}$

$\Delta = \{A, B, C\}$

Step 4 Example

Eg - 0101

$(q_0, 0101) \Rightarrow \delta(q_0, 0101)$ C

$\delta(q_0, 101)$ C

$\delta(q_1, 01)$ C

$\delta(q_2, 1)$ C

$\delta(q_3, \varepsilon)$ A

④ Design a moore m/c to generate 1's complement of given binary no.

⇒ **Step 1** Theory

To design a moore m/c to generate 1's complement of given binary no

Step 2 Logic

For 1's complement if input is 0 then o/p will be 1.

If I/P is 1 then o/p will be 0

(or)

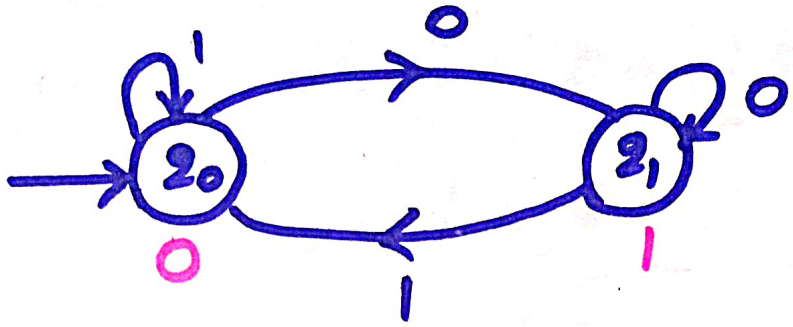
Next I/P as 0, sends the m/c to state Q_1 with o/p of 1 (complement of 0 is 1)

Next I/P as 1, sends the m/c to state Q_0 with o/p of 0.

(complement of 1 is 0)

Step 3 Implementation

State Transition Diagram



State Transition Table

δ :

Q \ Σ	0	1
→ z_0	z_1	z_0
z_1	z_1	z_0

λ :

	output
→ z_0	0
z_1	1

5 - tuples

$M = (Q, z_0, \Sigma, \Delta, \delta, \lambda)$

$Q = \{z_0, z_1\}$

$\Sigma = \{0, 1\}$

$\Delta = \{0, 1\}$

$z_0 = z_0$

Step 4 Example

eg - 101

$(q_0, 101) \Rightarrow \delta(q_0, 101)\ 0$

$\delta(q_0, 01)\ 0$

$\delta(q_1, 1)\ 1$

$\delta(q_0, \varepsilon)\ 0$

010

$\delta(q_0, 010) \Rightarrow \delta(q_0, 010)_0$

$\delta(q_1, 10)_1$

$\delta(q_0, 0)_0$

$\delta(q_1, \varepsilon)_1$

Mealy Machine

- In mealy mlc the output symbol is associated with transition
- In mealy mlc if the length of I/P sequence is n then length of output sequence is also n.
- It does not give the output on ε.
- It is represented by 6-tuples as follows.

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

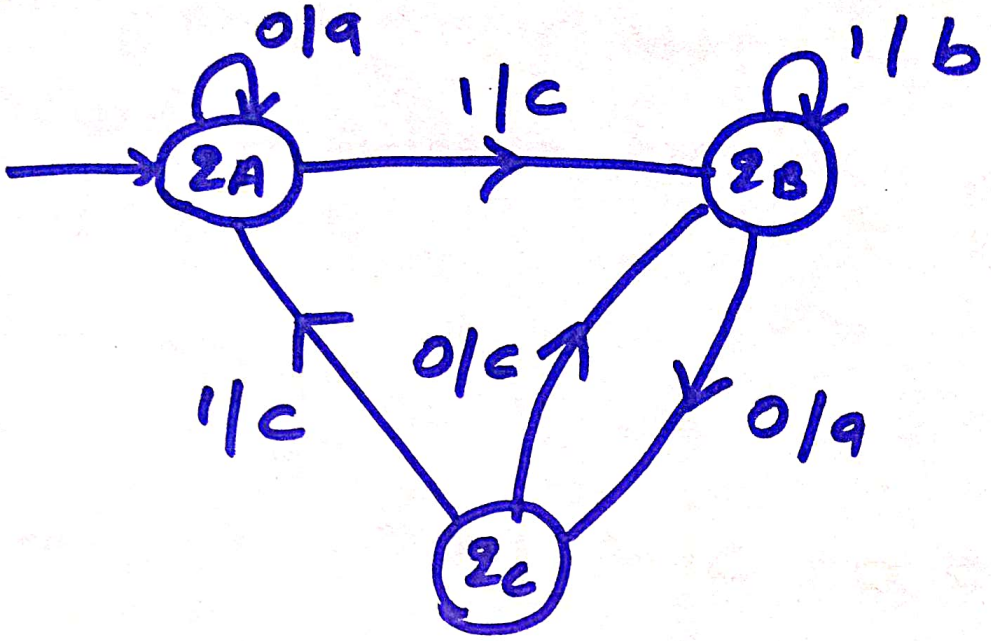
where

$Q \rightarrow$ Is finite set of states
$\Sigma \rightarrow$ Is finite set of I/P symbols
$\Delta \rightarrow$ Is finite o/p alphabets
$\delta \rightarrow$ Is state transition function, $\delta : Q \times \Sigma \rightarrow Q$
$\lambda \rightarrow$ Machine function

$$Q \times \Sigma \rightarrow \Delta$$

$q_0 \rightarrow$ Initial state

eg -



δ :

Q \ Σ	0	1
→ q_A	q_A	q_B
q_B	q_C	q_B
q_C	q_B	q_A

λ :

Q \ Σ	0	1
→ q_A	a	c
q_B	a	b
q_C	c	c

Example – 1010

$(q_A, 1010) \Rightarrow \delta(q_A, 1010)$ c

$\delta(q_B, 010)$ a

$\delta(q_C, 10)$ c

$\delta(q_A, 0)$ a

$\delta(q_A, \varepsilon)$

① Design Mealy MIC to o/p remainder when binary no. is divided by 4.

⇒ **Step 1** Theory

To design Mealy MIC to o/p remainder when binary no. is divided by 4

Step 2 Logic

Consider the following states

$Z_0 \rightarrow$	0	Remainder state	binary 00
$Z_1 \rightarrow$	1	"	01
$Z_2 \rightarrow$	2	"	10
$Z_3 \rightarrow$	3	"	11

	Q \ Σ	0	1
OR →	Z_0	00 % 4 = 0 (Z_0)	01 % 4 = 1 (Z_1)
1R	Z_1	10 = 2 % 4 = 2 (Z_2)	11 = 3 % 4 = 3 (Z_3)
2R	Z_2	100 = 4 % 4 = 0 (Z_0)	101 = 5 % 4 = 1 (Z_1)
3R	Z_3	110 = 6 % 4 = 2 (Z_2)	111 = 7 % 4 = 3 (Z_3)

δ :

Q \ Σ	0	1
→ q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_0	q_1
q_3	q_2	q_3

λ :

Q \ Σ	0	1
→ q_0	0R	1R
q_1	2R	3R
q_2	0R	1R
q_3	2R	3R

Minimization of Moore & Mealy machine -

In moore & mealy m/c states can be merged if

[all the states have same transition] and [output symbol associated with the transition is same]

After merging

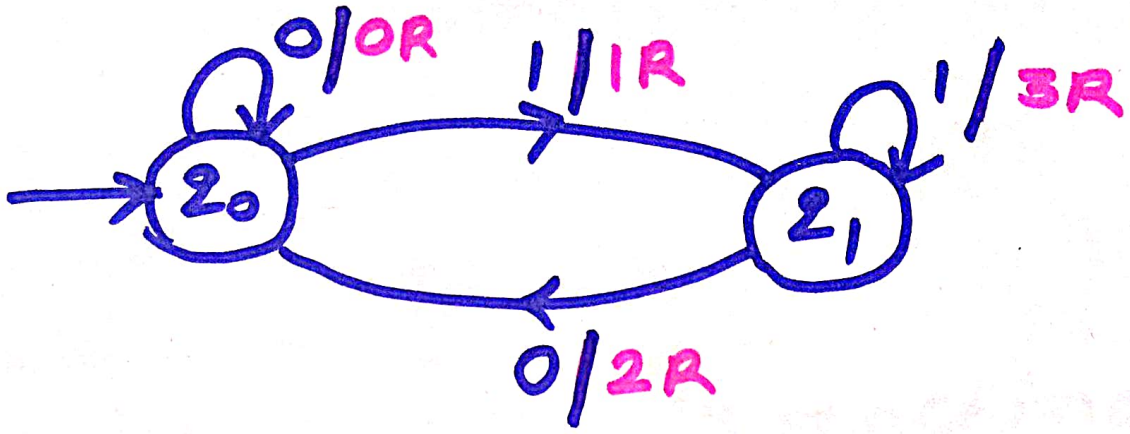
δ :

Q \ Σ	0	1
→ q_0	q_0	q_1
q_1	q_0	q_1

λ :

Q \ Σ	0	1
→ q_0	0R	1R
q_1	2R	3R

Step 3 Implementation



6 - tuples

$$M = (Q, \Sigma, \delta, \Delta, \lambda)$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$q_0 = q_0$$

$$\Sigma = \{0, 1\}$$

$$\Delta = \{0R, 1R, 2R, 3R\}$$

Step-4 Example

eg. 1010

$(q_0, 1010) \Rightarrow \delta(q_0, 1010)$ 1R

$\delta(q_1, 010)$ 2R

$\delta(q_0, 10)$ 1R

$\delta(q_1, 0)$ 2R

$\delta(q_0, \varepsilon)$

② Design a mealy machine to find 2's complement of a given binary no.

⇒ **Step 1** Theory

To design a mealy machine to find 2's complement of a given binary no.

Step 2 Logic

For designing 2's complement of binary no we assume that I/P is read from LSB to MSB.

suppose ① Given no - 1011

1's complement → 0100

2's complement → 0100
+ 1
0101

② Given no - 1 0 1 0

1's complement → 0 1 0 1

2's complement → 0 1 0 1

+ 1

0 1 1 0

To get o/p, we keep binary no as it is until we read first 1.

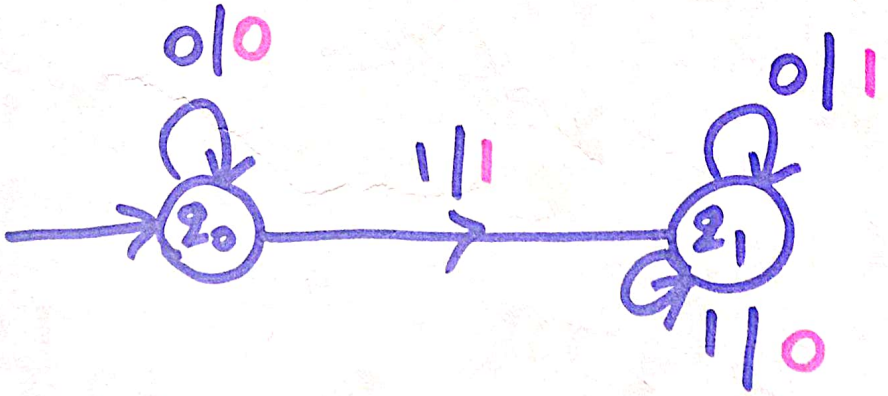
Keep that 1 as it is & then change remaining 1's by 0's & 0's by 1's

Consider the following states

① z_0

② z_1

Step 3 Implementation



δ :

Q \ Σ	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_1	q_1

λ :

Q \ Σ	0	1
$\rightarrow q_0$	0	1
q_1	1	0

6 tuples

$M = (Q, q_0, \Sigma, \delta, \Delta, \lambda)$

$Q = \{q_0, q_1\}$

$q_0 = q_0$

$\Sigma = \{0, 1\}$

$\Delta = \{0, 1\}$

Step 4 Example

① 0101

$(q_0, 0101) \Rightarrow \delta(q_0, 0101)$ ①

$\delta(q_1, 1010)$ ①

$\delta(q_1, 01)$ ①

$\delta(q_1, 0)$ ①

$\delta(q_1, \varepsilon)$ ✓

0101

1's 1010

2's + 1

1011

③ Construct Mealy M/c that produces 1's complement of any binary I/P string.

⇒ **Step 1** Theory

To construct Mealy M/c that produces 1's complement of any binary I/P string.

Step 2 Logic

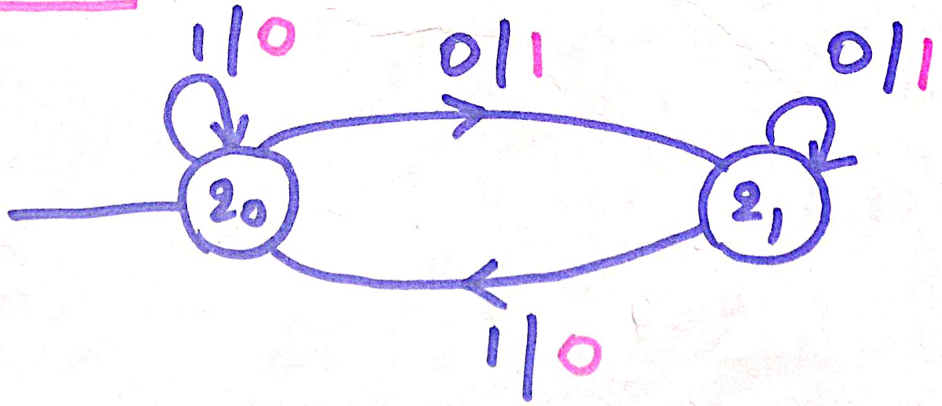
For 1's complement if input is 0 then o/p will be 1 & If I/P is 1 then o/p will be 0

Consider the following states -

① Z_0

② Z_1

Step 3 Implementation



δ :

q \ Σ	0	1
→ q_0	q_1	q_0
q_1	q_1	q_0

λ :

q \ Σ	0	1
→ q_0	1	0
q_1	1	0

6 - tuples

$M = (Q, q_0, \Sigma, \delta, \Delta, \lambda)$

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$

$\Delta = \{0, 1\}$

Step 4 Example

eg - 1010

$(q_0, 1010) \Rightarrow \delta(\underline{q_0}, 101\underline{0})$ 1

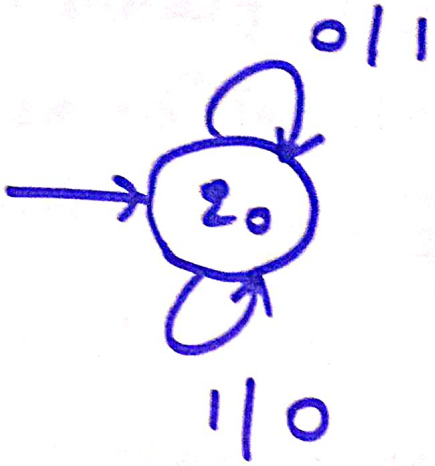
$\delta(\underline{q_1}, 10\underline{1})$ 0

$\delta(\underline{q_0}, 1\underline{0})$ 1

$\delta(\underline{q_1}, \underline{1})$ 0

$\delta(q_0, \varepsilon)$ ↑ 0101

(2)



δ :

q \ Σ	0	1
$\rightarrow q_0$	q_0	q_0

λ :

q \ Σ	0	1
$\rightarrow q_0$	1	0

eg- 1010

$\delta(q_0, 1010) \Rightarrow \delta(q_0, 101\underline{0})1$

$\delta(q_0, 10\underline{1})0$

$\delta(q_0, 1\underline{0})1$

$\delta(q_0, \underline{1})0$

$\delta(q_0, \varepsilon)$

$\uparrow 0101$

④ Design Mealy M/c to O/P
A if I/P ends in 101
B otherwise over {0, 1}

⇒ **Step 1** Theory

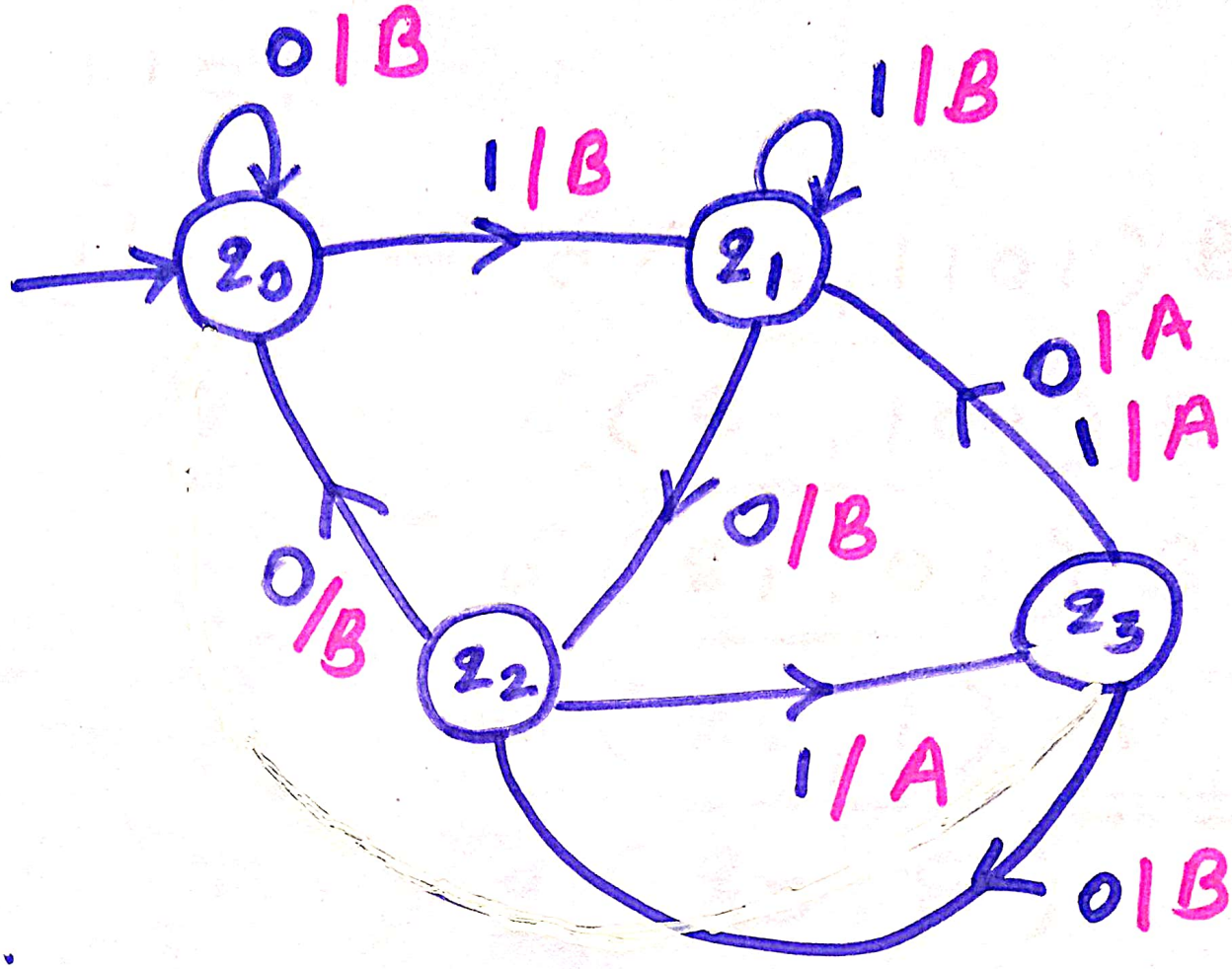
To design Mealy M/c to O/P
A if I/P ends in 101
B if otherwise over {0, 1}

Step 2 Logic

$q_0 \rightarrow$ I/P ends with 0
$q_1 \rightarrow$ I/P ends with 1
$q_2 \rightarrow$ I/P ends with 10
$q_3 \rightarrow$ I/P ends with 101

	q \ Σ	0	1
0	$\rightarrow q_0$	0<u>0</u> (q_0)	0<u>1</u> (q_1)
1	q_1	<u>10</u> (q_2)	<u>11</u> (q_1)
10	q_2	10<u>0</u> (q_0)	<u>101</u> (q_3)
101	q_3	101<u>0</u> (q_2)	101<u>1</u> (q_1)

Step 3 Implementation



δ :

Q \ Σ	0	1
→z_0	z_0	z_1
z_1	z_2	z_1
z_2	z_0	z_3
z_3	z_2	z_1

λ :

Q \ Σ	0	1
→z_0	B	B
z_1	B	B
z_2	B	A
z_3	B	B

Step 4 Example

eg - 1101

$(q_0, 1101) \Rightarrow \delta(q_0, 1101)\ B$

$\delta(q_1, 101)\ B$

$\delta(q_1, 01)\ B$

$\delta(q_2, 1)\ A$

$\delta(q_3, \varepsilon)$

eg - 1001

$(q_0, 1001) \Rightarrow \delta(q_0, 1001)\ B$

$\delta(q_1, 001)\ B$

$\delta(q_2, 01)\ B$

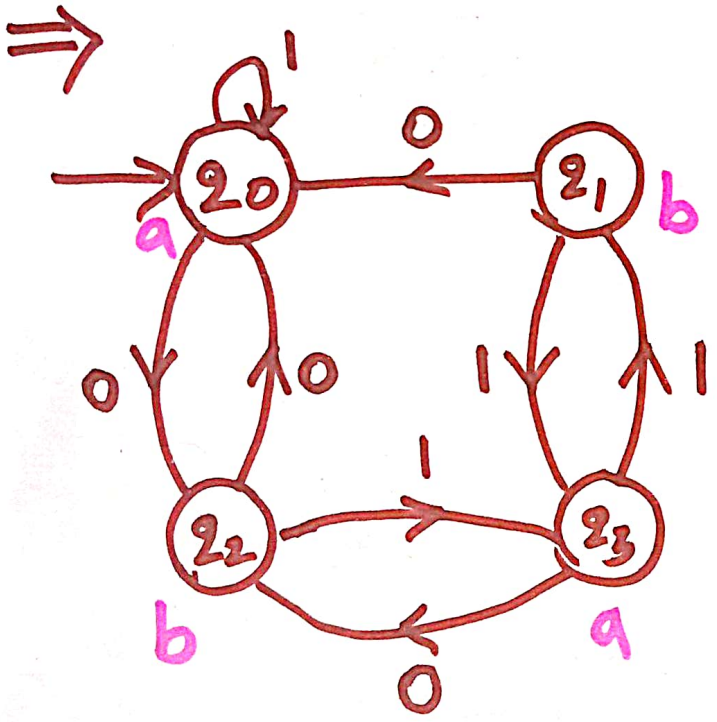
$\delta(q_0, 1)\ B$

$\delta(q_1, \varepsilon)$

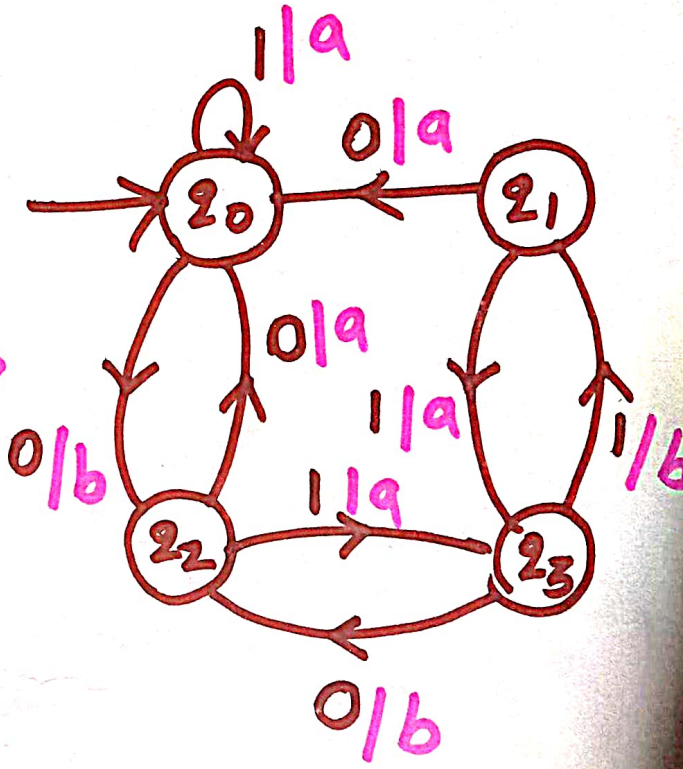
Conversion of Moore M|C to Mealy M|C -

Assign o|p symbol associated with the state to all of its incoming transitions

① convert the following moore m|c to mealy m|c.



Moore M|C

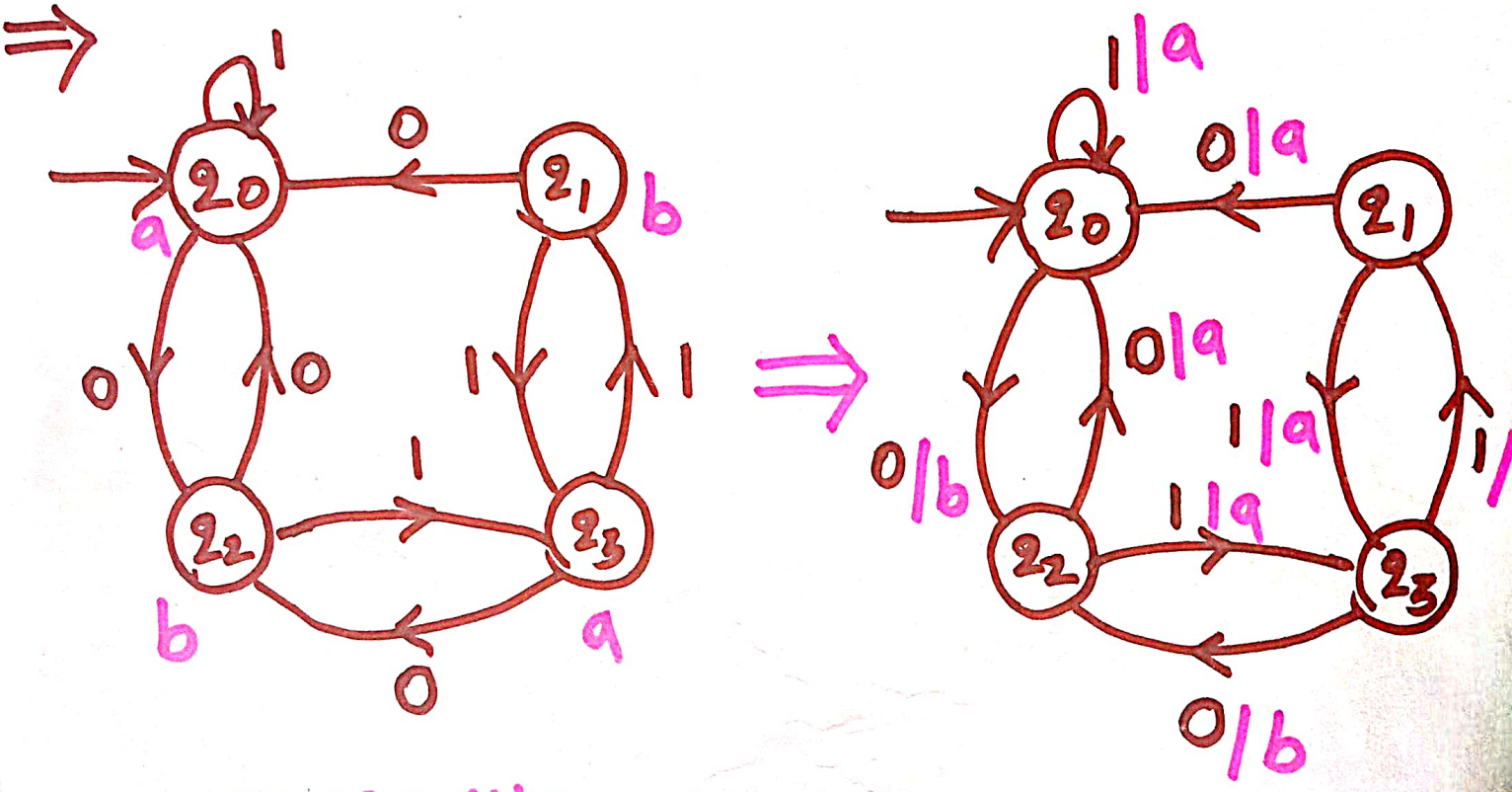


Mealy M|C

Conversion of Moore M|C to Mealy M|C -

Assign o|p symbol associated with the state to all of its incoming transitions

① convert the following moore m|c to mealy m|c.



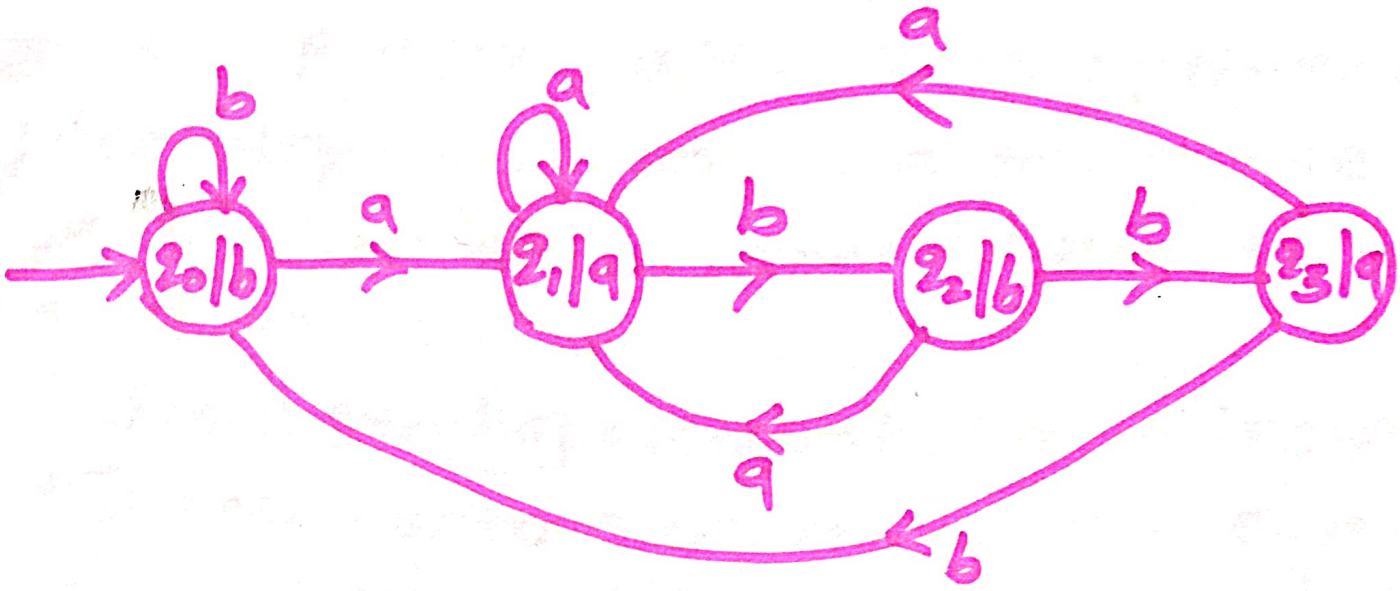
δ :

Q \ Σ	0	1
$\rightarrow q_0$	q_2	q_0
q_1	q_0	q_3
q_2	q_0	q_3
q_3	q_2	q_1

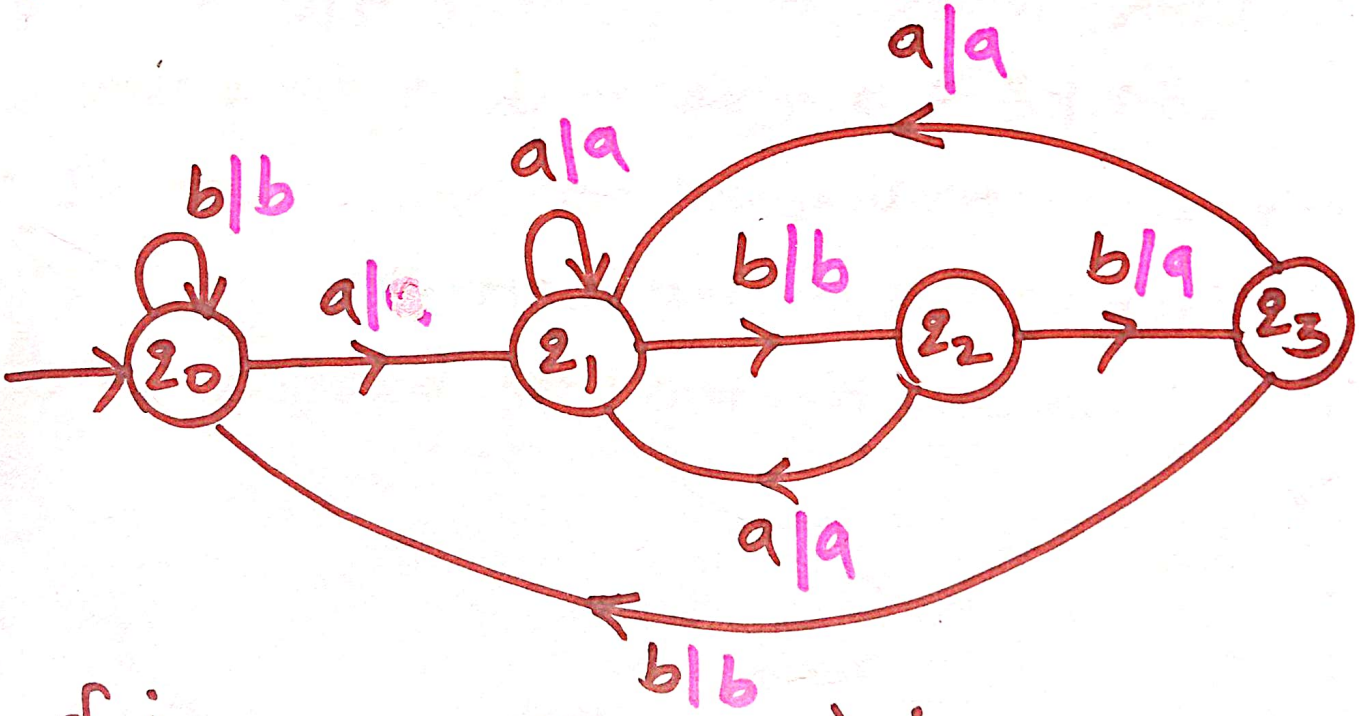
λ :

Q \ Σ	0	1
$\rightarrow q_0$	b	a
q_1	a	a
q_2	a	a
q_3	b	b

(2) convert following moore machine to mealy machine.



⇒



δ :

q \ Σ	a	b
→ z_0	z_1	z_0
z_1	z_1	z_2
z_2	z_1	z_3
z_3	z_1	z_0

λ :

q \ Σ	a	b
→ z_0	a	b
z_1	a	b
z_2	a	a
z_3	a	b

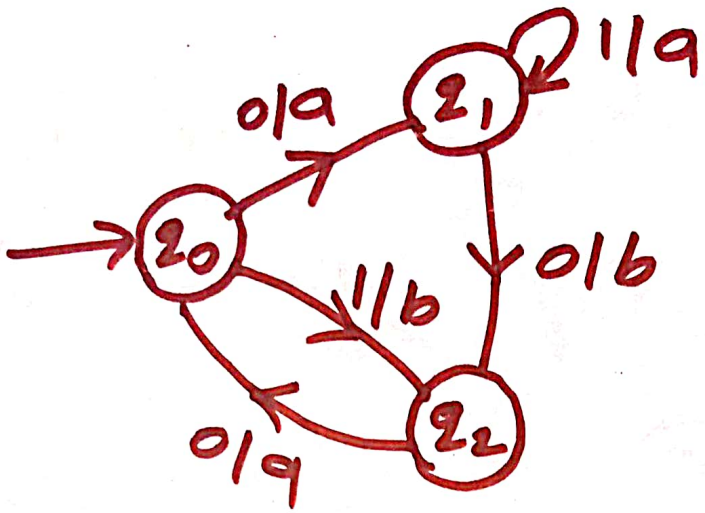
Conversion of Mealy M/c to Moore Machine -

- If o/p symbols associate with the incoming transition to a state are same then assign that symbol to that state.
- If o/p symbol associated with incomin transition to a state are not same then split that state as many times as the o/p symbol with each state producing a different o/p symbol.
- If there are no incoming transition to a state then any o/p symbol can be assigned to that state.

(1)

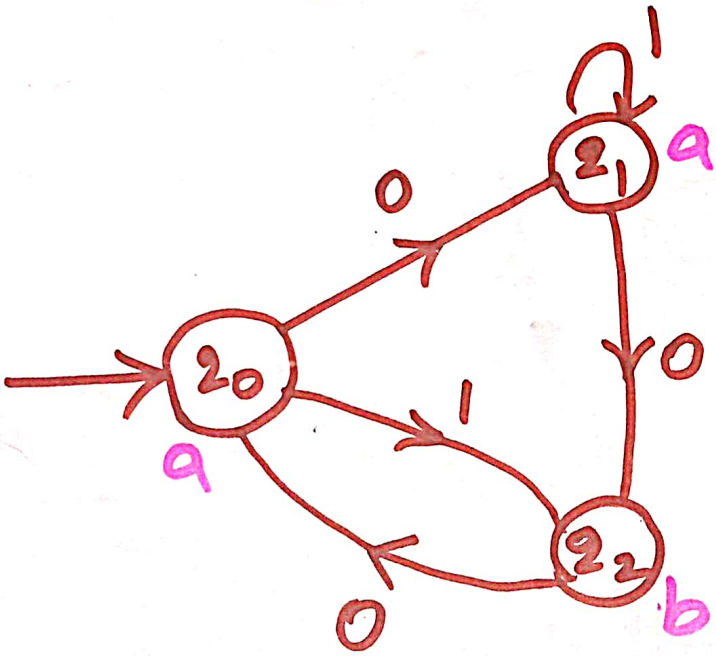
① Convert the following Mealy Machine to Moore machine.

⇒



⇒

Mealy Machine



Moore Machine

δ :

Σ / a	0	1
$\rightarrow z_0$	z_1	z_2
z_1	z_2	z_1
z_2	z_0	ϕ

λ :

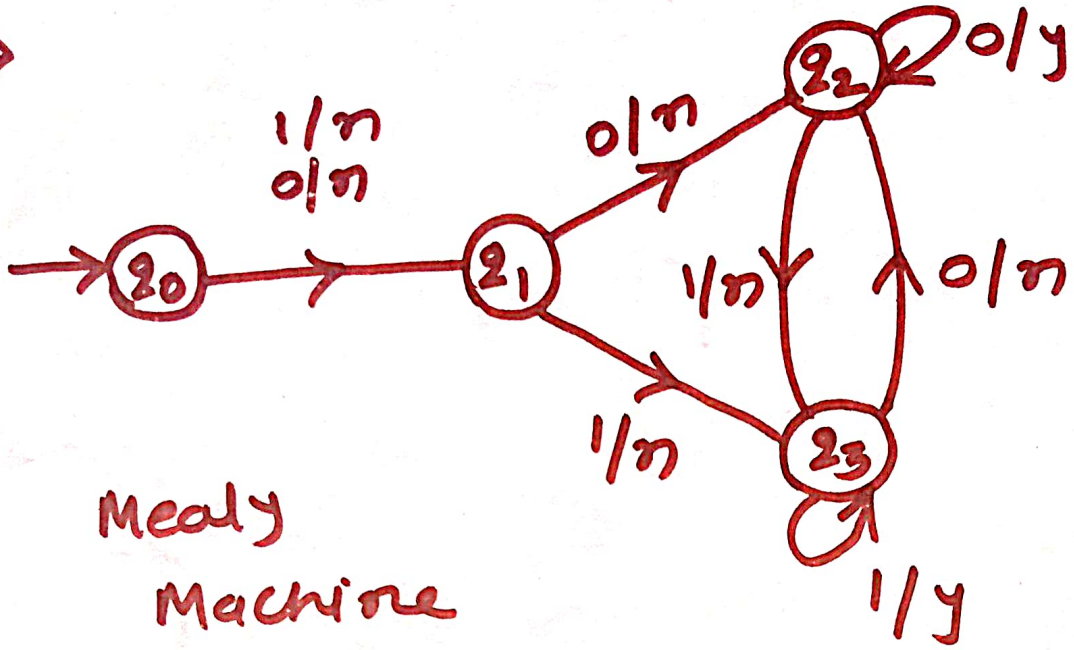
$\lambda(z_0) = a$

$\lambda(z_1) = a$

$\lambda(z_2) = b$

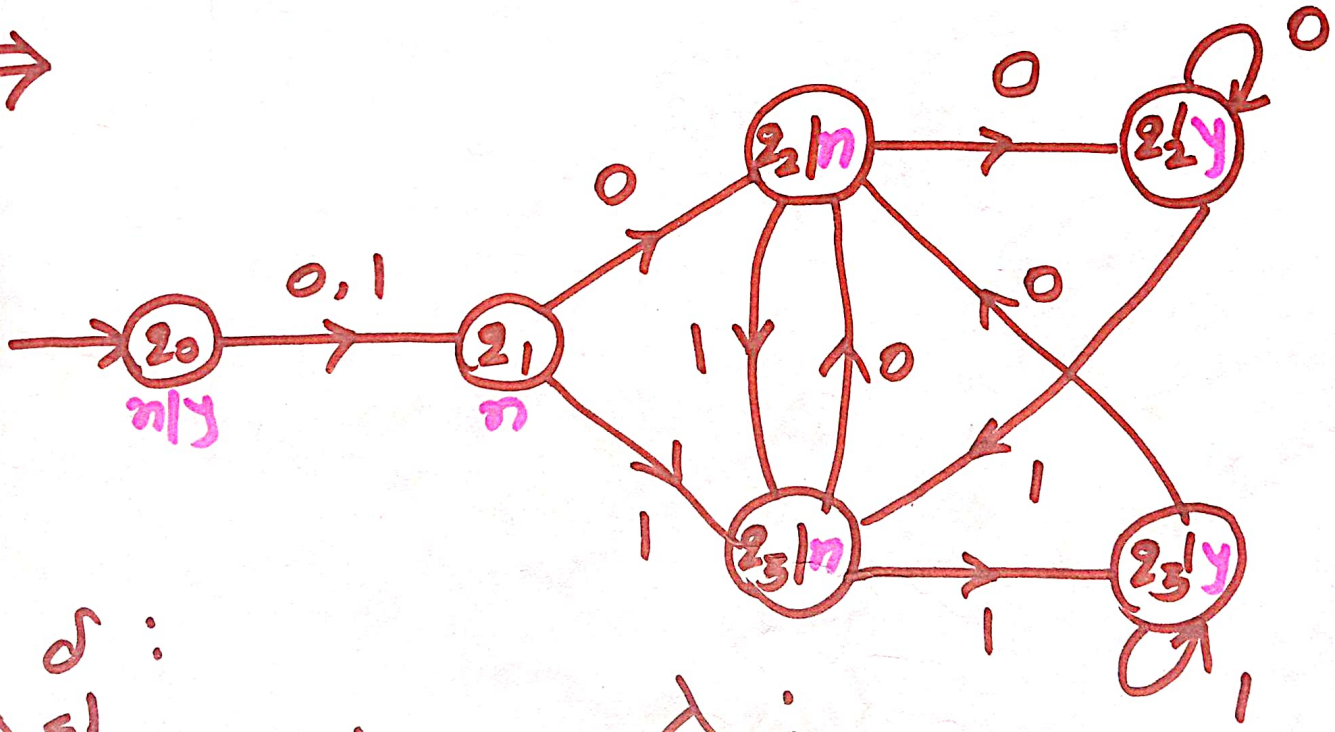
② convert the following Mealy Machine to Moore Machine.

⇒



Mealy Machine

⇒



δ :

q / Σ	0	1
→ q_0	q_1	q_1
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_2	q_3

λ :

$\lambda(q_0) = n/y$

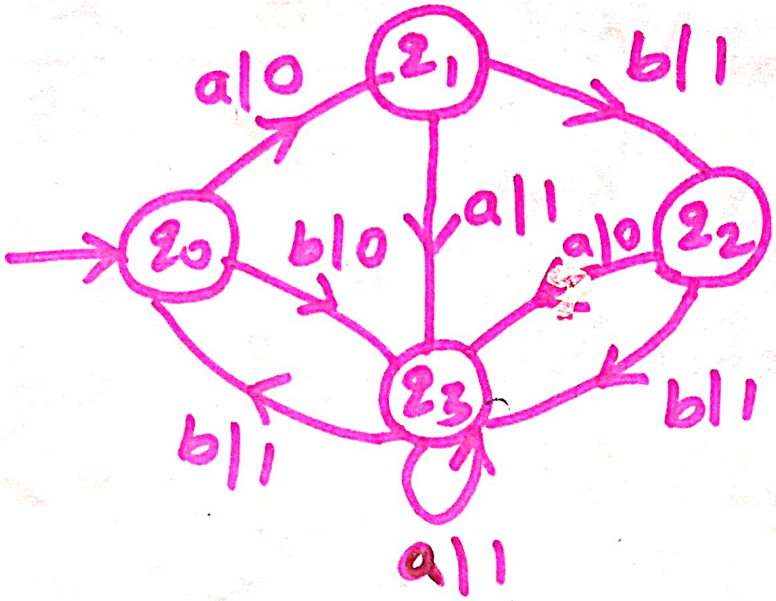
$\lambda(q_1) = n$

$\lambda(q_2) = n/y$

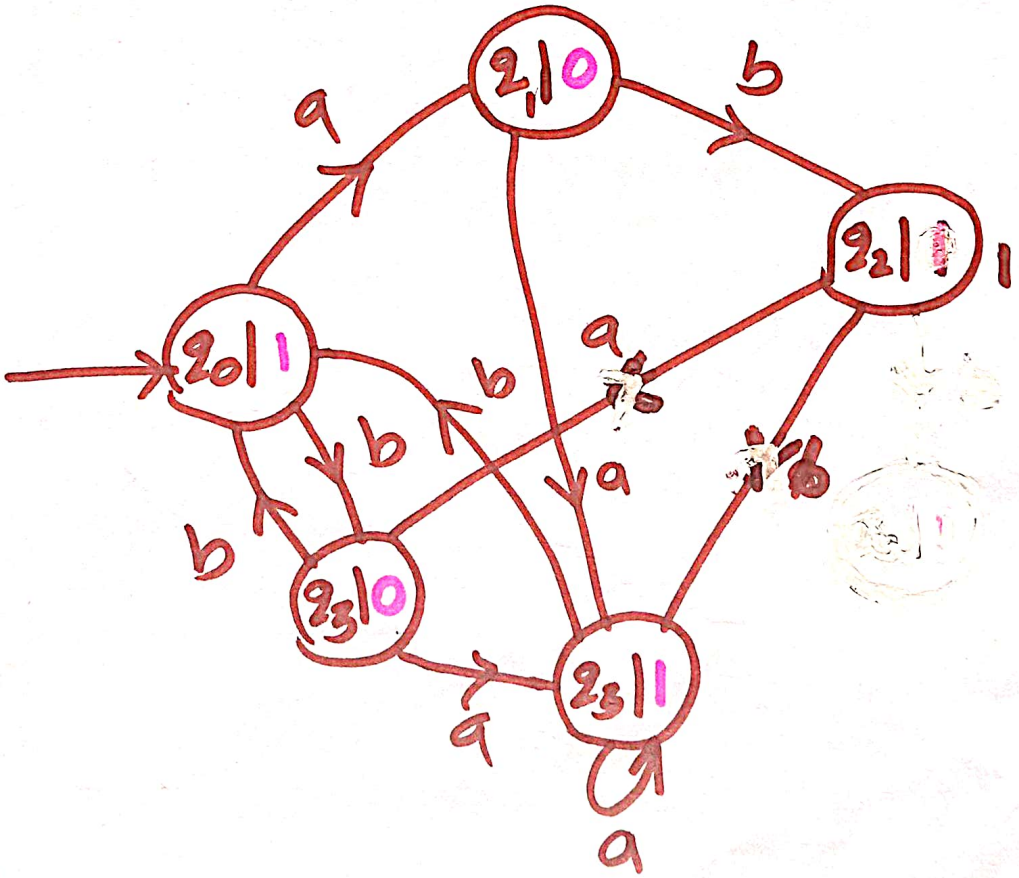
$\lambda(q_3) = n/y$

③ Convert the following Mealy M/C to Moore M/C

10)



⇒



δ :

$Q \backslash \Sigma$	a	b
$\rightarrow q_0$	q_1	q_3
q_1	q_3	q_2
q_2	q_3	q_3
q_3	q_3	ϕ

λ :

$\lambda(q_0) = 1$

$\lambda(q_1) = 0$

$\lambda(q_2) = 1$

$\lambda(q_3) = 0/1$

Difference between Moore M/C & Mealy M/C

Moore Machine	Mealy Machine
① In moore M/C State gives o/p	① In mealy M/C transition gives o/p
② The o/p mapping is defined as $\lambda : Q \rightarrow \Delta$	② The o/p mapping is defined as $\lambda : Q \times \Sigma \rightarrow \Delta$
③ In moore m/c if length of I/P sequence is n then the length of o/p sequence is $n+1$.	③ In mealy m/c if the length of I/P sequence is n then the length of o/p sequence is also n
④ Moore m/c gives o/p on Σ.	④ Mealy m/c does not give o/p on Σ.
⑤ While converting moore m/c to mealy m/c, the no. of States remains the same	⑤ While converting mealy m/c to moore m/c, the no. of States might or might not increases

Minimization of DFA -

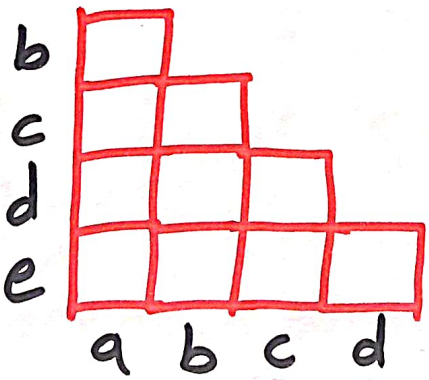
(Based on Myhill Nerode Theorem)
(M-N Theorem)

STEP 1

Remove those states which are not reachable from the start state.

STEP 2

Create a table such that rows & columns would be identified by state numbers.
Suppose a, b, c, d & e are the states then the table would be as shown below



STEP 3

All those cells in which one state is final & other is non final should be marked with symbol 'X'
(Since such states represent distinguishable states)

STEP 4

For the unmarked cells perform the following check

Let (P, Q) be an unmarked cell & let 'a' be some I/P symbol.

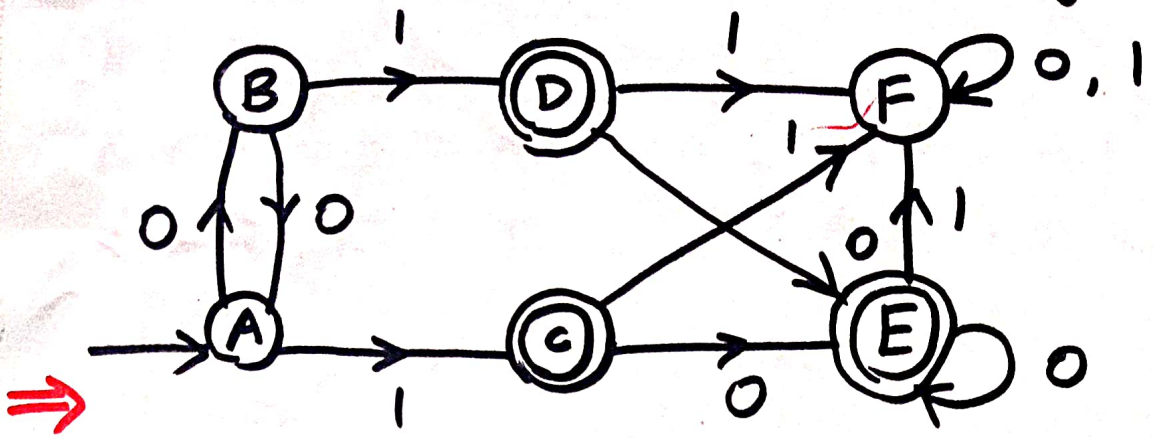
- If $\delta(P, a) = Q$ and $\delta(Q, a) = S$ then mark (P, Q) with symbol '✗' if (Q, S) is marked.
  else create list (Q, S) = (P, Q)
- If $\delta(P, a) = Q$ and $\delta(Q, a) = Q$ then no decision can be taken with respect to this input

If the cell (P, Q) is unmarked then repeat the step with another input symbol (if available)

STEP 5

All those cells which are unmarked should be merged into single state reply (since states represents equivalent states)

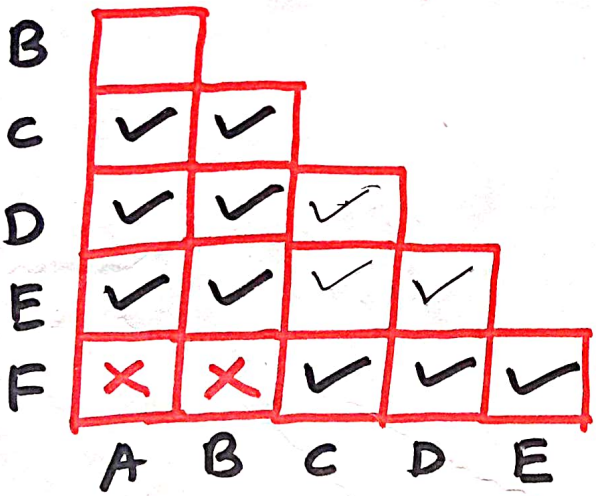
① Minimize the following DFA



STEP 1

All states are reachable from the start state.

STEP 2



STEP 3

The cells in which one state is final & other is non final is marked as ✓ in the table

STEP 4

① (B, A)

δ(B, 0) = A
δ(B, 1) = D

δ(A, 0) = B
δ(A, 1) = C

② (D, C)

δ(D, 0) = E
δ(D, 1) = F

δ(C, 0) = E
δ(C, 1) = F

③ (E, C)

δ(E, 0) = E
δ(E, 1) = F

δ(C, 0) = E
δ(C, 1) = F

④ (E, D)

δ(E, 0) = E
δ(E, 1) = F

δ(D, 0) = E
δ(D, 1) = F

⑤ (F, A)

δ(F, 0) = F
δ(F, 1) = F

δ(A, 0) = B
δ(A, 1) = C

⑥ (F, B)

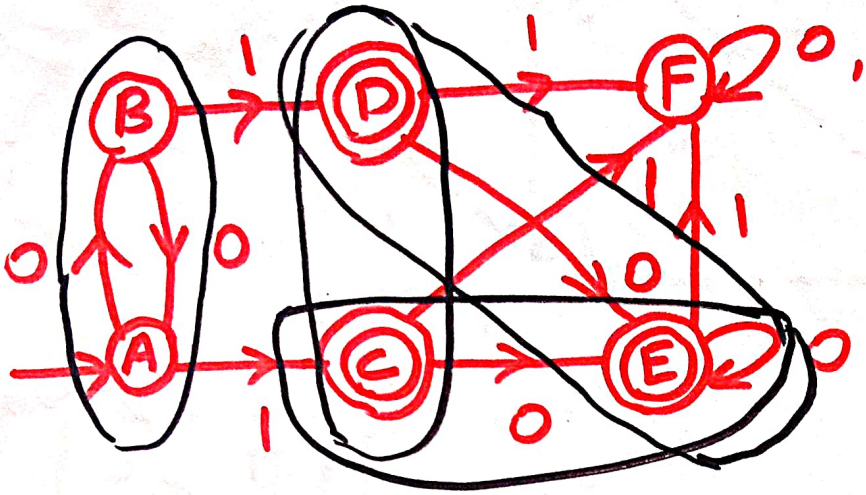
δ(F, 0) = F
δ(F, 1) = F

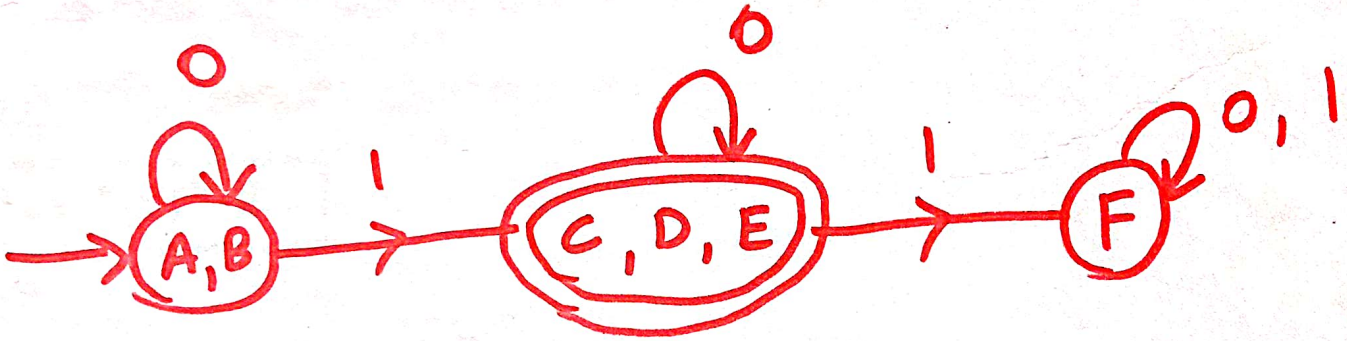
δ(B, 0) = A
δ(B, 1) = D

STEP 5

Unmarked states are as follows

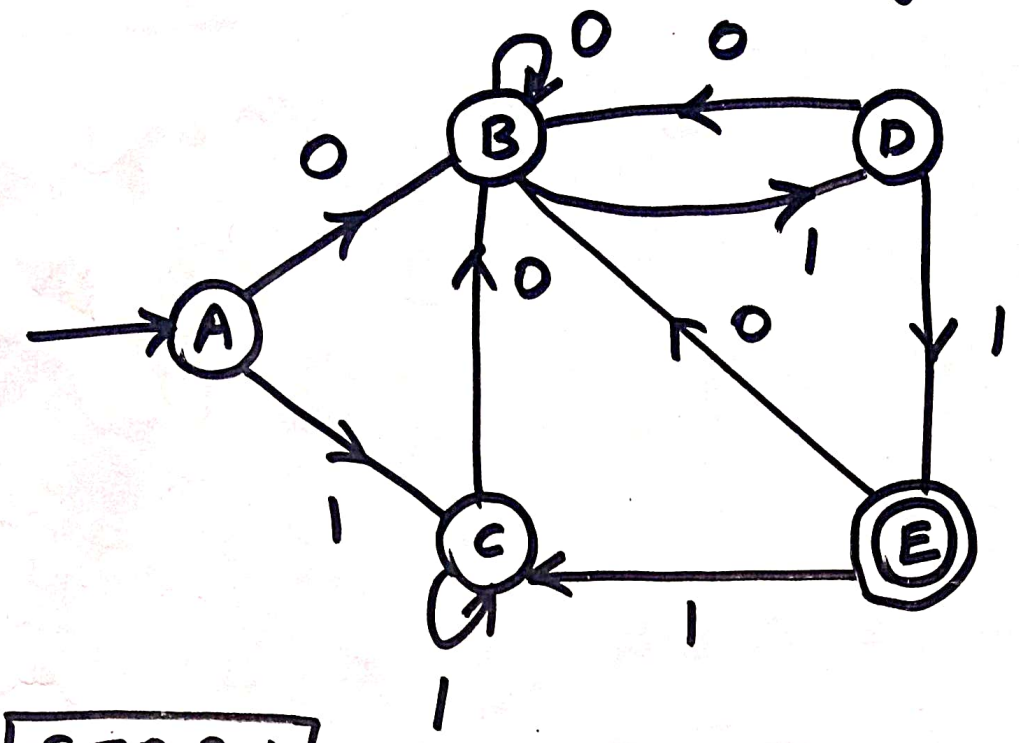
(A, B), (DC), (E, C), (E, D)





This is minimized DFA

) Minimize the following DFA



⇒ **STEP 1**

All the states are reachable from the start state

STEP 2

	A	B	C	D
B	X			
C		X		
D	X	X	X	
E	✓	✓	✓	✓

STEP 3

The cells in which one state is final & other is non final is marked as ✓ in the table

STEP 4

① (B, A)

$\delta(B, 0) = B$

$\delta(B, 1) = D$

$\delta(A, 0) = B$

$\delta(A, 1) = C$

② (C, A)

$\delta(C, 0) = B$

$\delta(C, 1) = C$

$\delta(A, 0) = B$

$\delta(A, 1) = C$

③ (C, B)

$\delta(C, 0) = B$

$\delta(C, 1) = C$

$\delta(B, 0) = B$

$\delta(B, 1) = D$

④ (D, A)

$\delta(D, 0) = B$

$\delta(D, 1) = E$

$\delta(A, 0) = B$

$\delta(A, 1) = C$

⑤ (D, B)

$\delta(D, 0) = B$

$\delta(D, 1) = E$

$\delta(B, 0) = B$

$\delta(B, 1) = D$

⑥ (D, C)

δ(D, 0) = B, δ(C, 0) = B

δ(D, 1) = E, δ(C, 1) = C

⑦ (B, A)

δ(B, 0) = B, δ(A, 0) = B

δ(B, 1) = D, δ(A, 1) = C

⑧ (C, A)

δ(C, 0) = B, δ(A, 0) = B

δ(C, 1) = C, δ(A, 1) = C

⑨ (C, B)

δ(C, 0) = B, δ(B, 0) = B

δ(E, 1) = C, δ(B, 1) = D

⑩ (C, A)

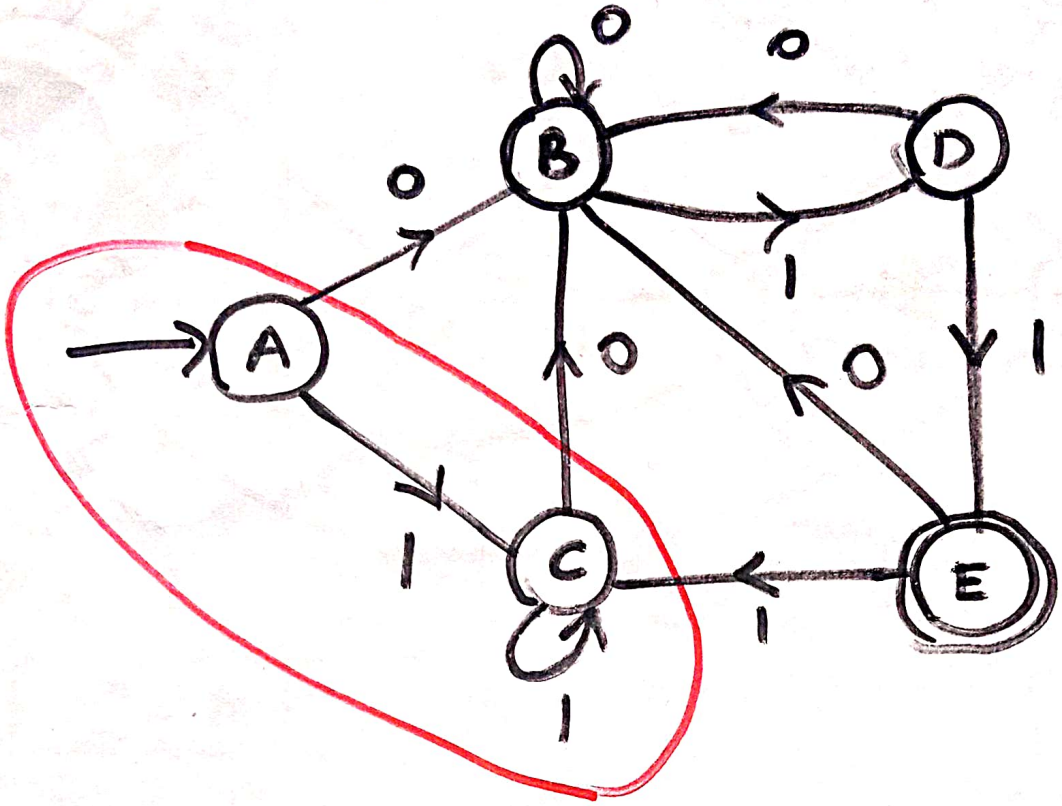
δ(C, 0) = B, δ(A, 0) = B

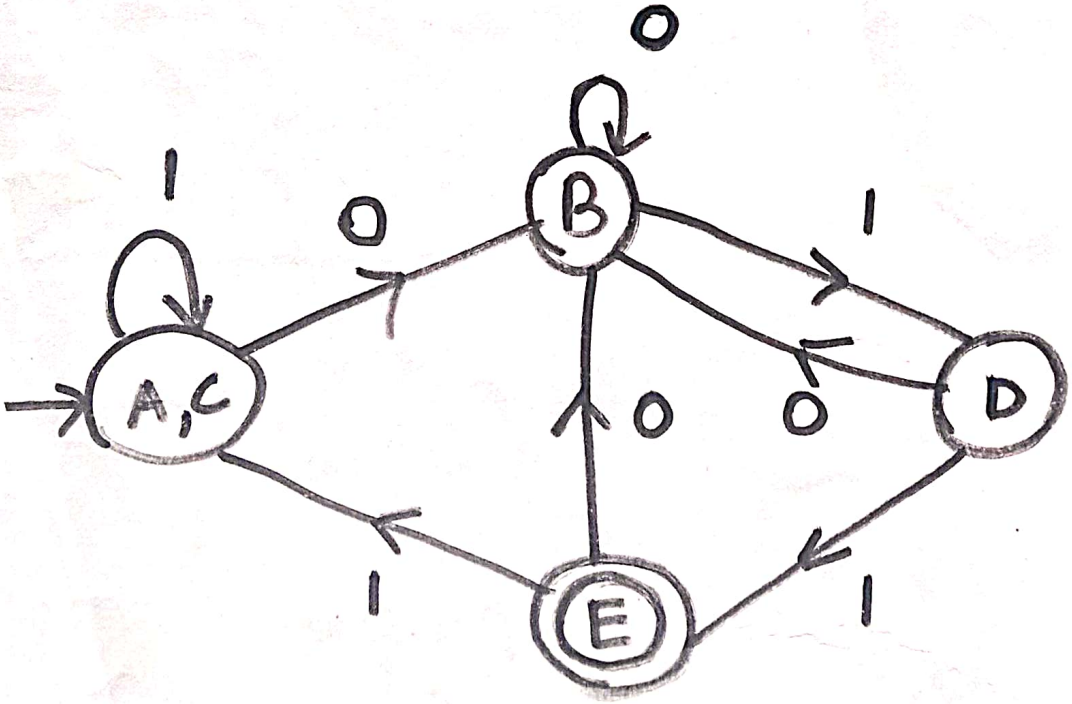
δ(C, 1) = C, δ(A, 1) = C

STEP 5

Unmarked states are as follows

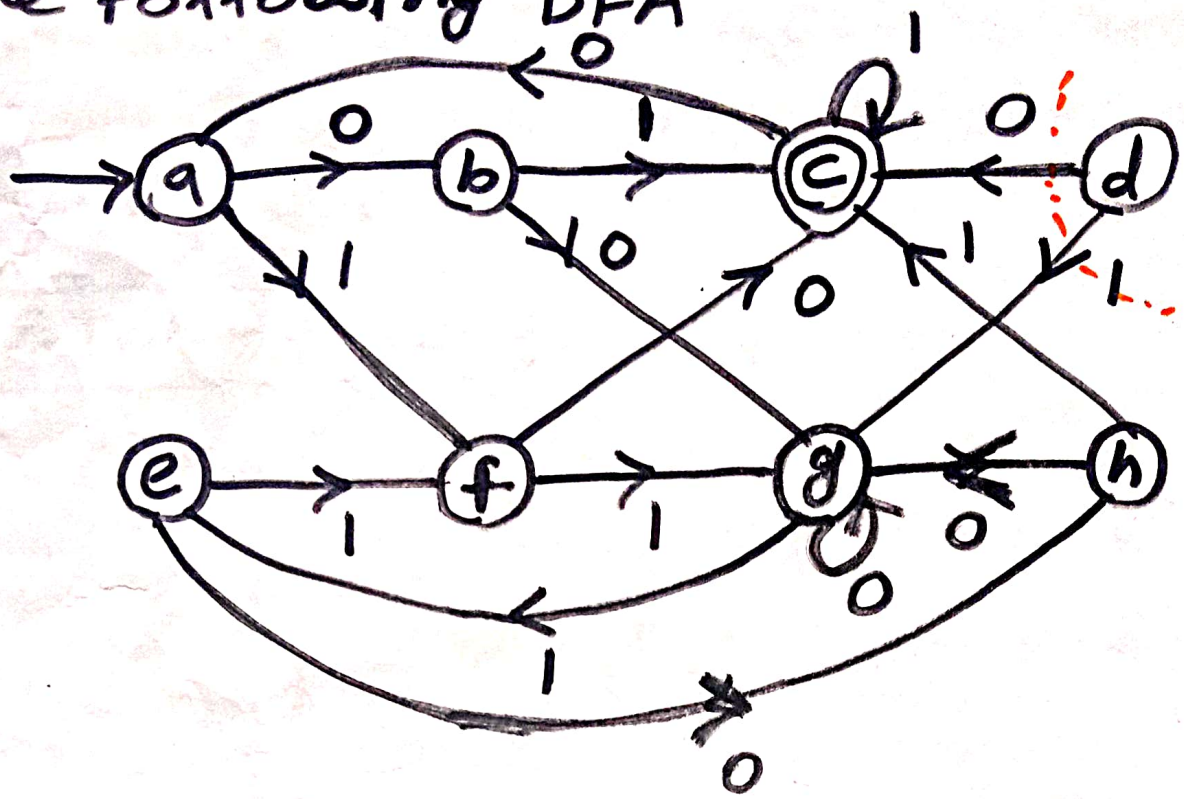
(A, C), B, D, E





This is minimized DFA

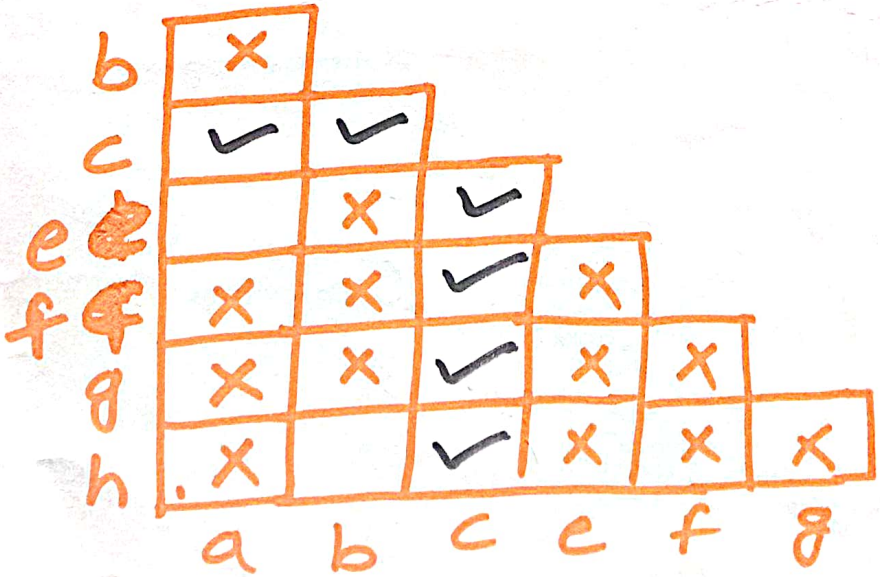
③ Construct the minimized DFA for the following DFA



⇒ **STEP 1**

Since 'd' is not reachable from the start state so we eliminate that state.

STEP 2-



STEP 3

The cells in which one state is final other is non-final is marked as X in table

STEP 4

① (b, a)

$\delta(b,0) = g$ $\delta(a,0) = b$

$\delta(b,1) = c$ $\delta(a,1) = f$

② (e, a)

$\delta(e,0) = h$ $\delta(a,0) = b$

$\delta(e,1) = f$ $\delta(a,1) = f$

③ (e, b)

$(e,0) = h$ $\delta(b,0) = g$

$(e,1) = f$ $\delta(b,1) = c$

④ (f, a)

$\delta(f,0) = c$ $\delta(a,0) = b$

$\delta(f,1) = g$ $\delta(a,1) = f$

⑤ (f, b)

$\delta(f,0) = c$ $\delta(b,0) = g$

$\delta(f,1) = g$ $\delta(b,1) = c$

⑥ (f, e)

$\delta(f,0) = c$ $\delta(e,0) = h$

$\delta(f,1) = g$ $\delta(e,1) = f$

⑦ (g, a)

$\delta(g,0) = g$ $\delta(a,0) = b$

$\delta(g,1) = e$ $\delta(a,1) = f$

⑧ (g, b)

$\delta(g,0) = g$ $\delta(b,0) = g$

$\delta(g,1) = e$ $\delta(b,1) = c$

(g, e)

$\delta(g,0) = g$ $\delta(e,0) = h$

$\delta(g,1) = e$ $\delta(e,1) = f$

⑩ (g, f)

$\delta(f,0) = g$ $\delta(f,0) = c$

$\delta(g,1) = e$ $\delta(f,1) = g$

(11) (h, a)

$\delta(h, 0) = g$ $\delta(a, 0) = b$

$\delta(h, 1) = c$ $\delta(a, 1) = f$

(12) (h, b)

$\delta(h, 0) = g$ $\delta(b, 0) = g$

$\delta(h, 1) = c$ $\delta(b, 1) = c$

(13) (h, e)

$\delta(h, 0) = g$ $\delta(e, 0) = h$

$\delta(h, 1) = c$ $\delta(e, 1) = f$

(14) (h, f)

$\delta(h, 0) = g$ $\delta(f, 0) = g$

$\delta(h, 1) = c$ $\delta(f, 1) = g$

(h, g)

$\delta(h, 0) = g$ $\delta(g, 0) = g$

$\delta(h, 1) = c$ $\delta(g, 1) = e$